

A

B

C

D

E

Nametable Mapping
File: nametable_mapping.kicad_sch
CPU Address Bus

Register Access
File: register_access.kicad_sch
PRG Banking
File: prg_banking.kicad_sch
PPU Address Bus
Reset Detection
File: reset_detection.kicad_sch
CPU Data Bus
File: cpu_data_bus.kicad_sch
PCM DAC
Scanline Detection
File: scanline_detection.kicad_sch
V-Split
File: v_split.kicad_sch
PPU Fetch Counting
File: ppu_fetch_counting.kicad_sch
CHR A0,A1,A2
File: chr_a0_a1_a2.kicad_sch
ExRAM Mode
File: exram_mode.kicad_sch
Fill Mode
Pulse Length Lookup Table
File: pulse_length_lookup_table.kicad_sch
Pulse Timer
File: pulse_timer.kicad_sch
Pulse Duty Envelope Volume
File: pulse_duty_envelope_volume.kicad_sch
Pulse DAC
File: Pulse DAC.kicad_sch
PPU Data Bus
CHR Bank Registers
File: chr_bank_registers.kicad_sch
CHR Bank Selection Logic
File: chr_bank_selection_logic.kicad_sch
CHR Address Bus 10 to 14
File: chr_address_bus_10_to_14.kicad_sch
CHR Address Bus 15 to 19
File: chr_address_bus_15_to_19.kicad_sch
Multiplier
File: multiplier.kicad_sch
Artifacts
File: artifacts.kicad_sch

+PCM_IRQ_If_Enabled{K114}
+Scanline_IRQ_Pin{X14}

K115
NAND.3

K116
INVERT

/IRQ_PIN

/Test_Mode_PIN_92

PinInv
INVERT

+Test_Mode_PIN_92{PinInv}

Test_Input_PIN_29

PinInv1
INVERT

/Test_Input_PIN_29{PinInv}

Test_Input_PIN_30

PinInv2
INVERT

/Test_Input_PIN_30{PinInv}

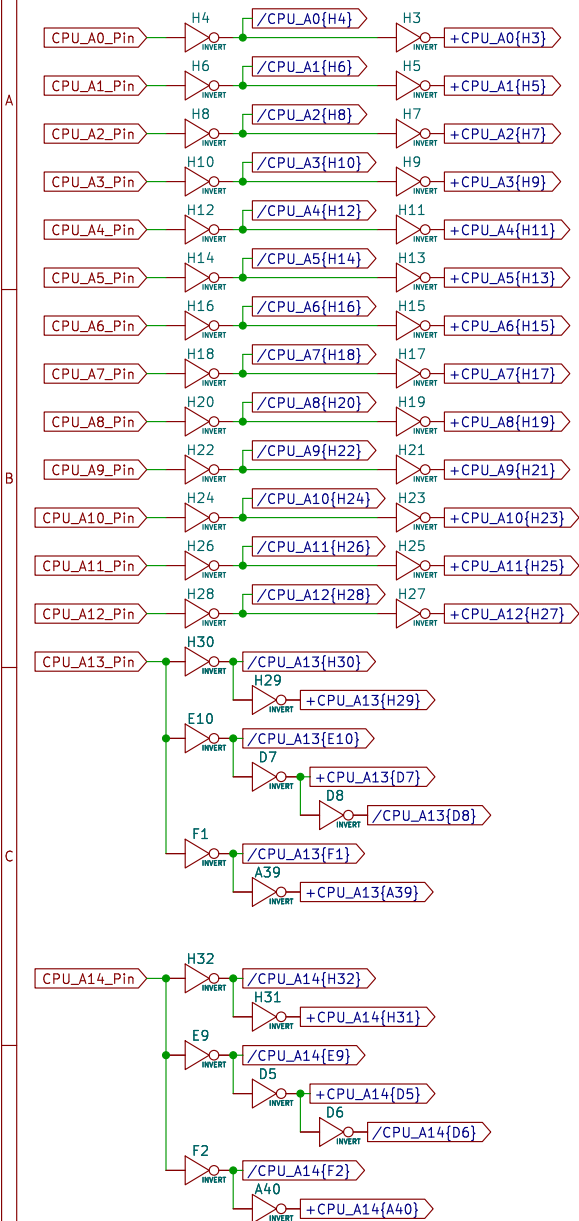
SL3_PIN_98

PinInv3
INVERT

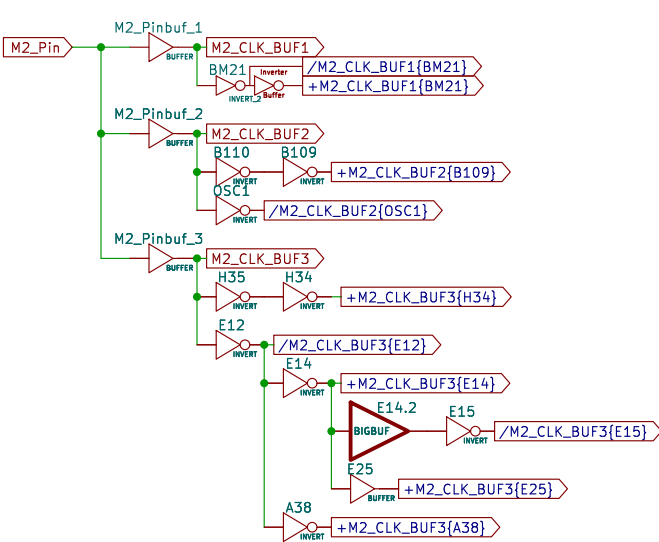
/SL3_PIN_98{PinInv}

Title:
Size: USLedger | Date: | Rev:
KiCad E.D.A. 9.0.7 | Id: 1/26

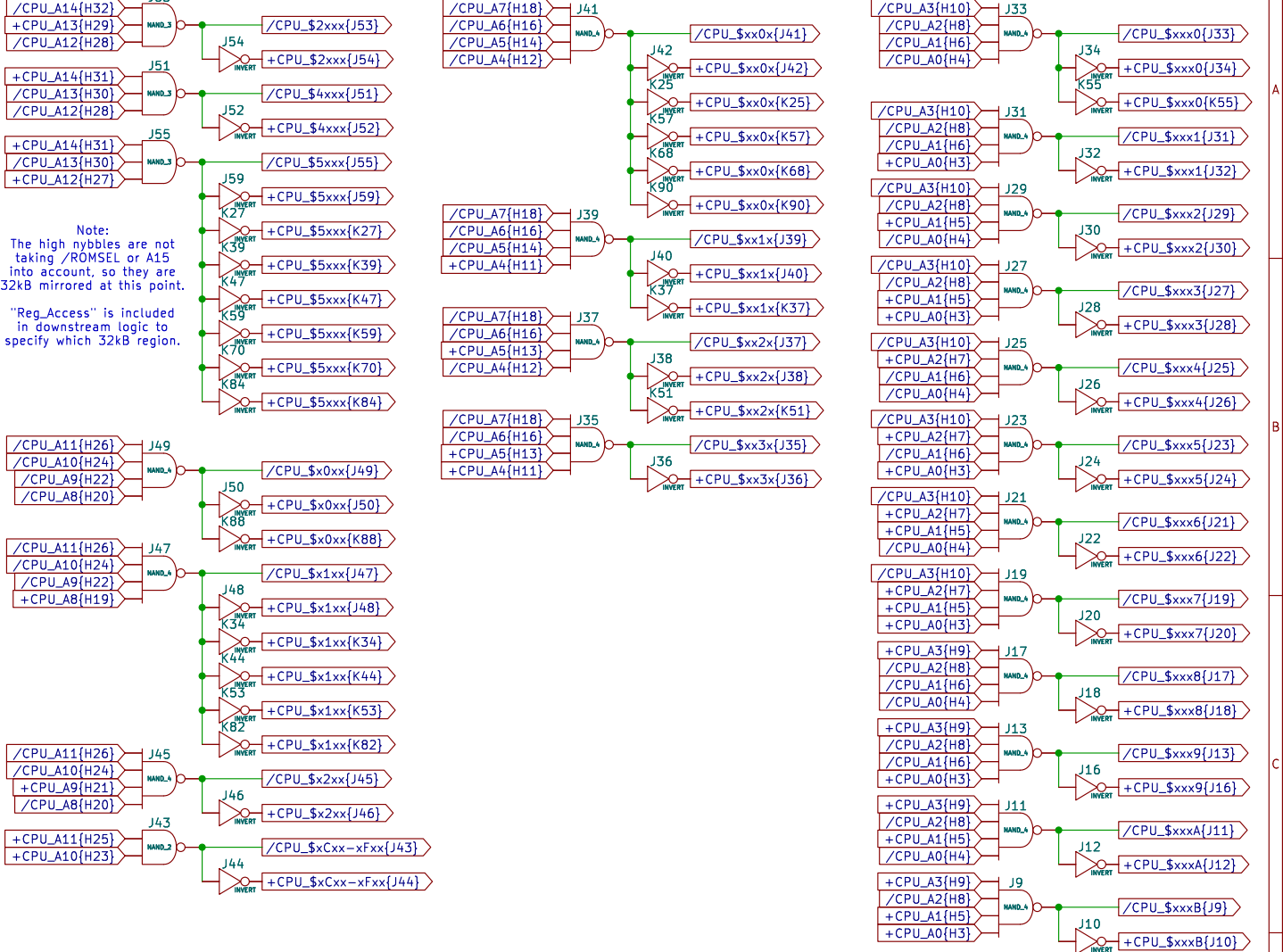
CPU Address Bus Buffering



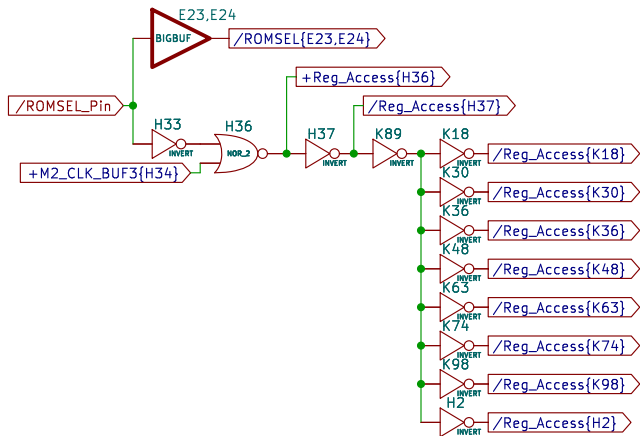
M2 Clock Buffering



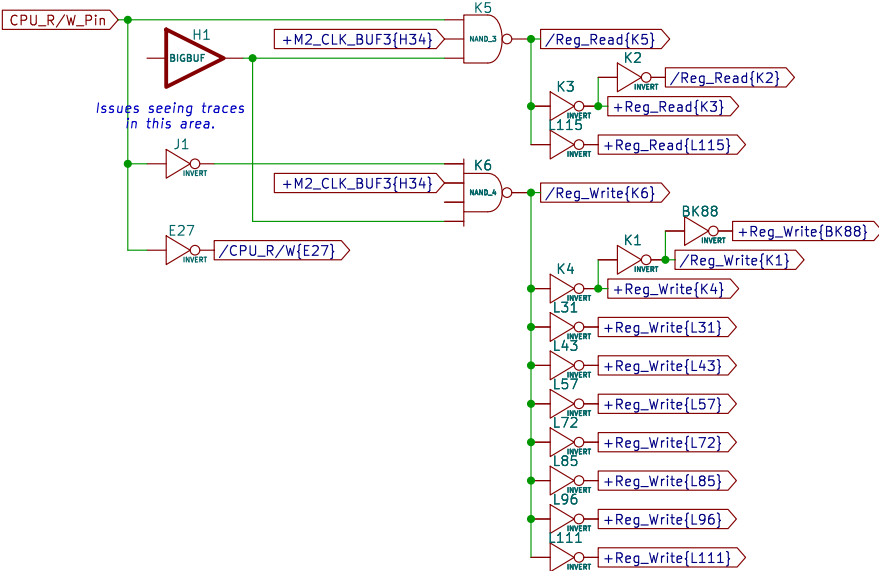
CPU Address Bus Nybble Decoding

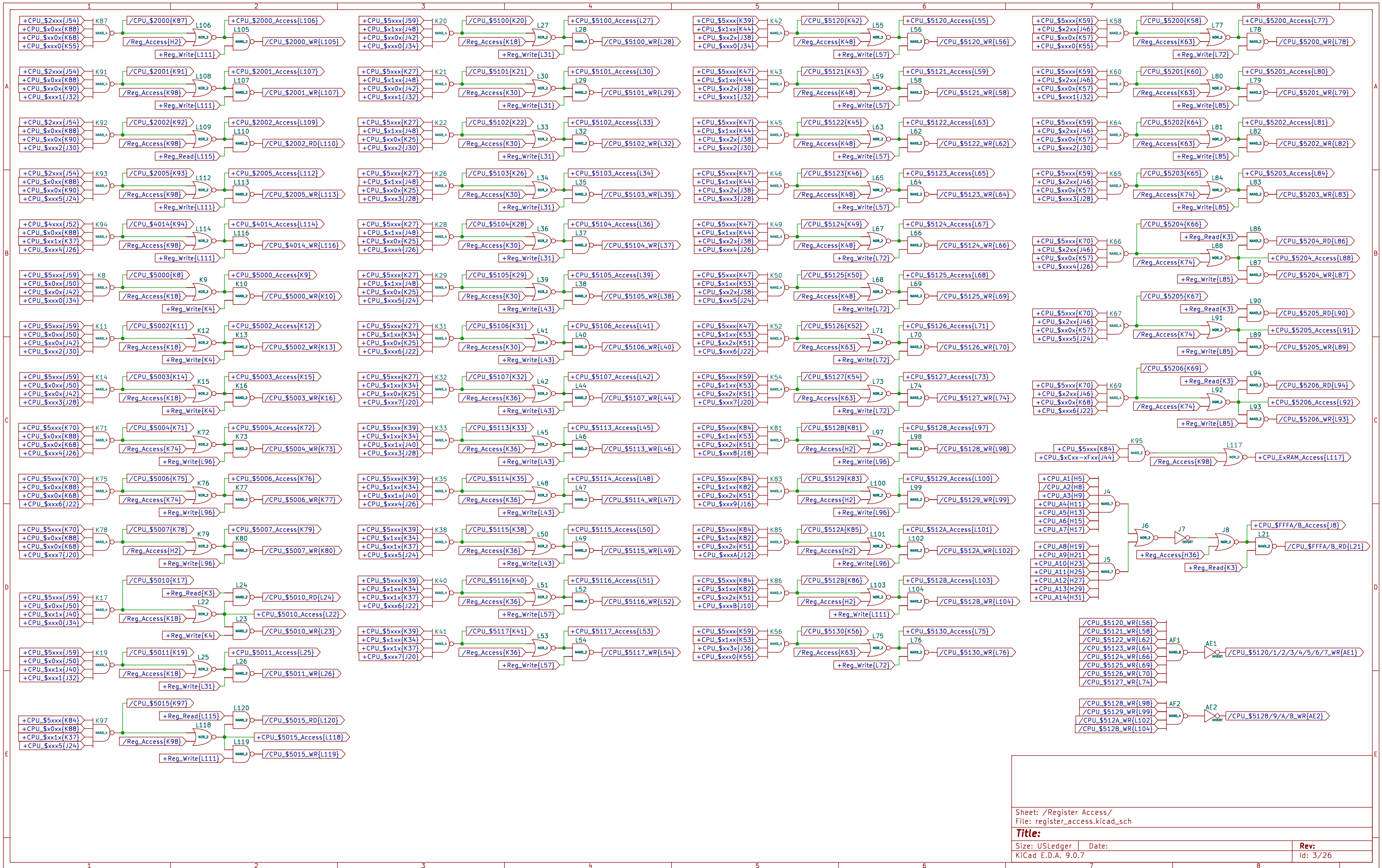


/ROMSEL, Register Access

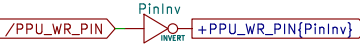
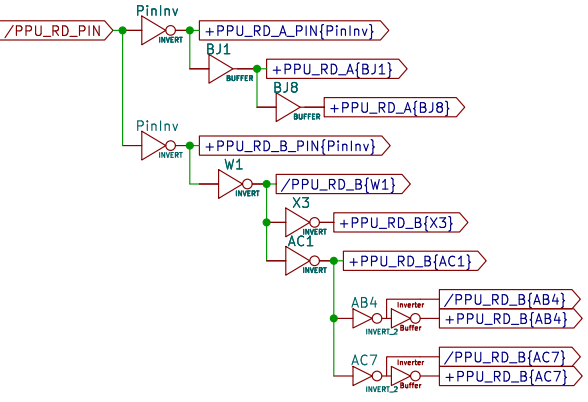
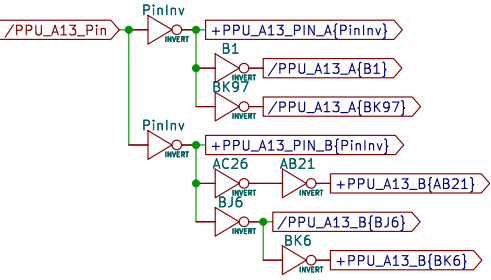
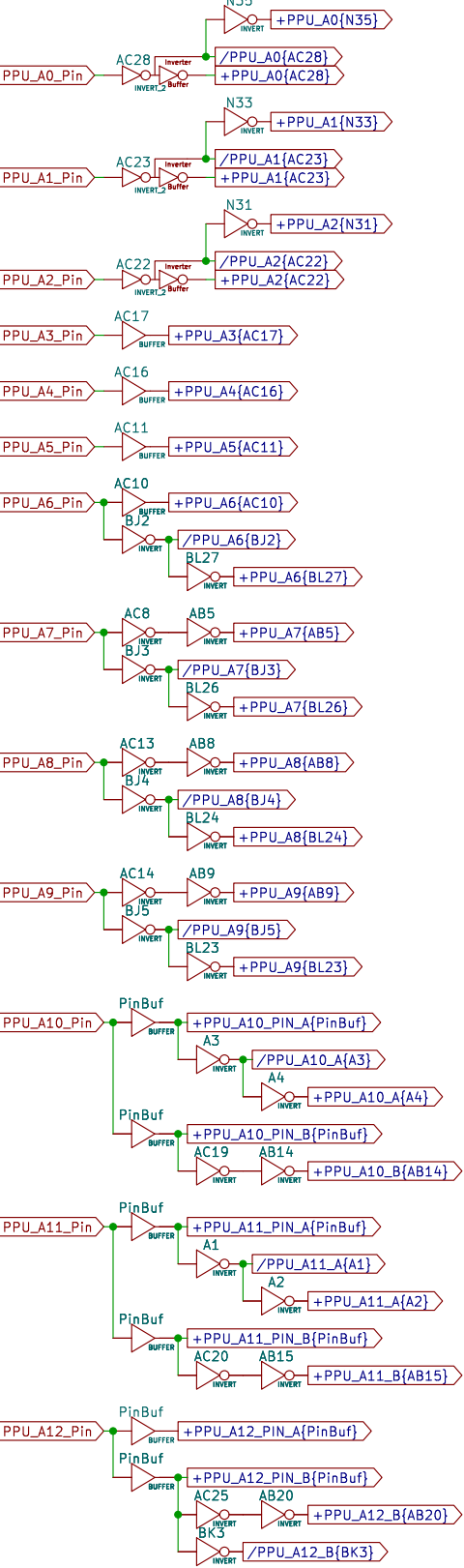


CPU R/W, Register Read/Write

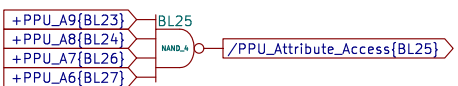


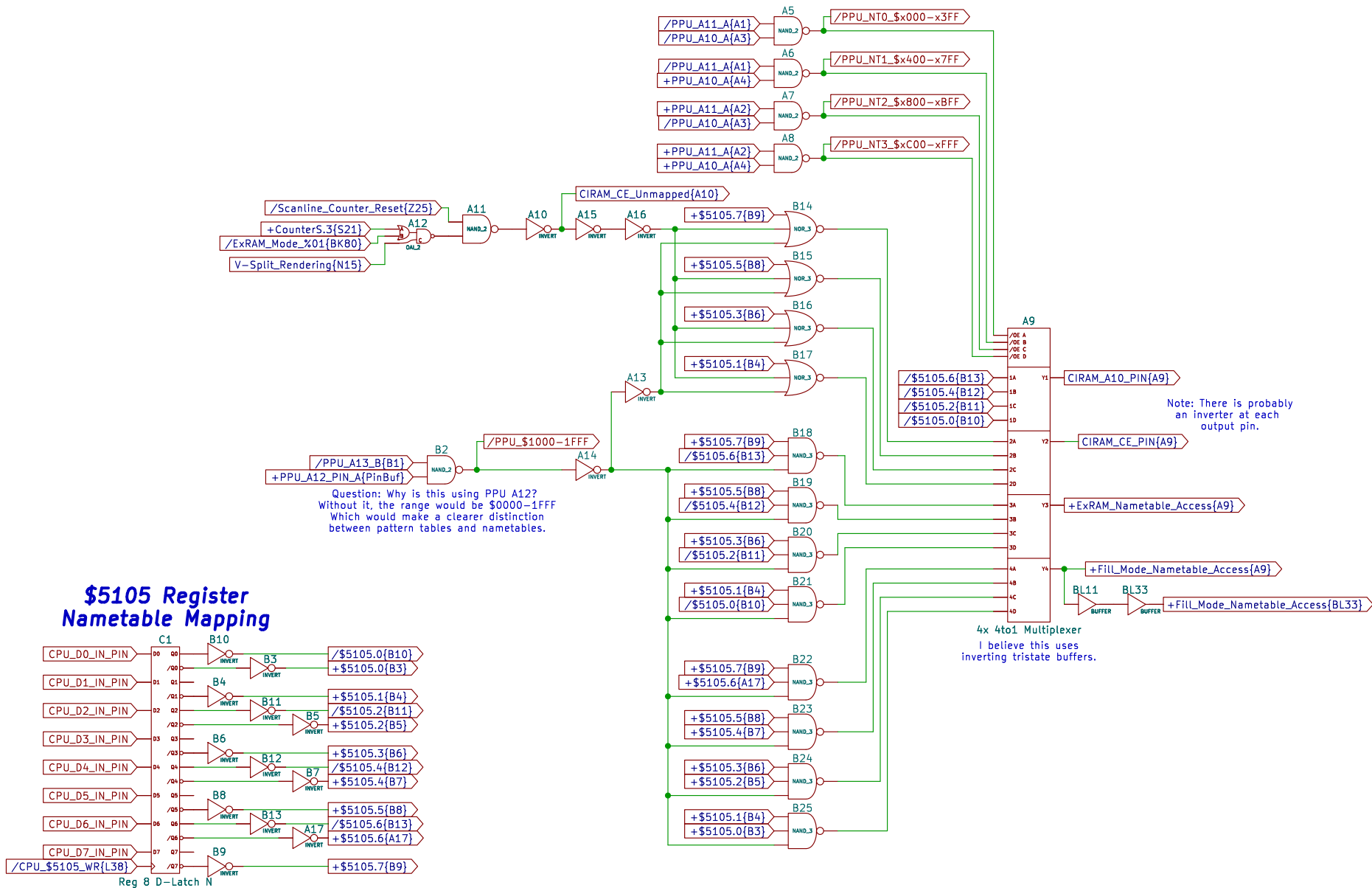


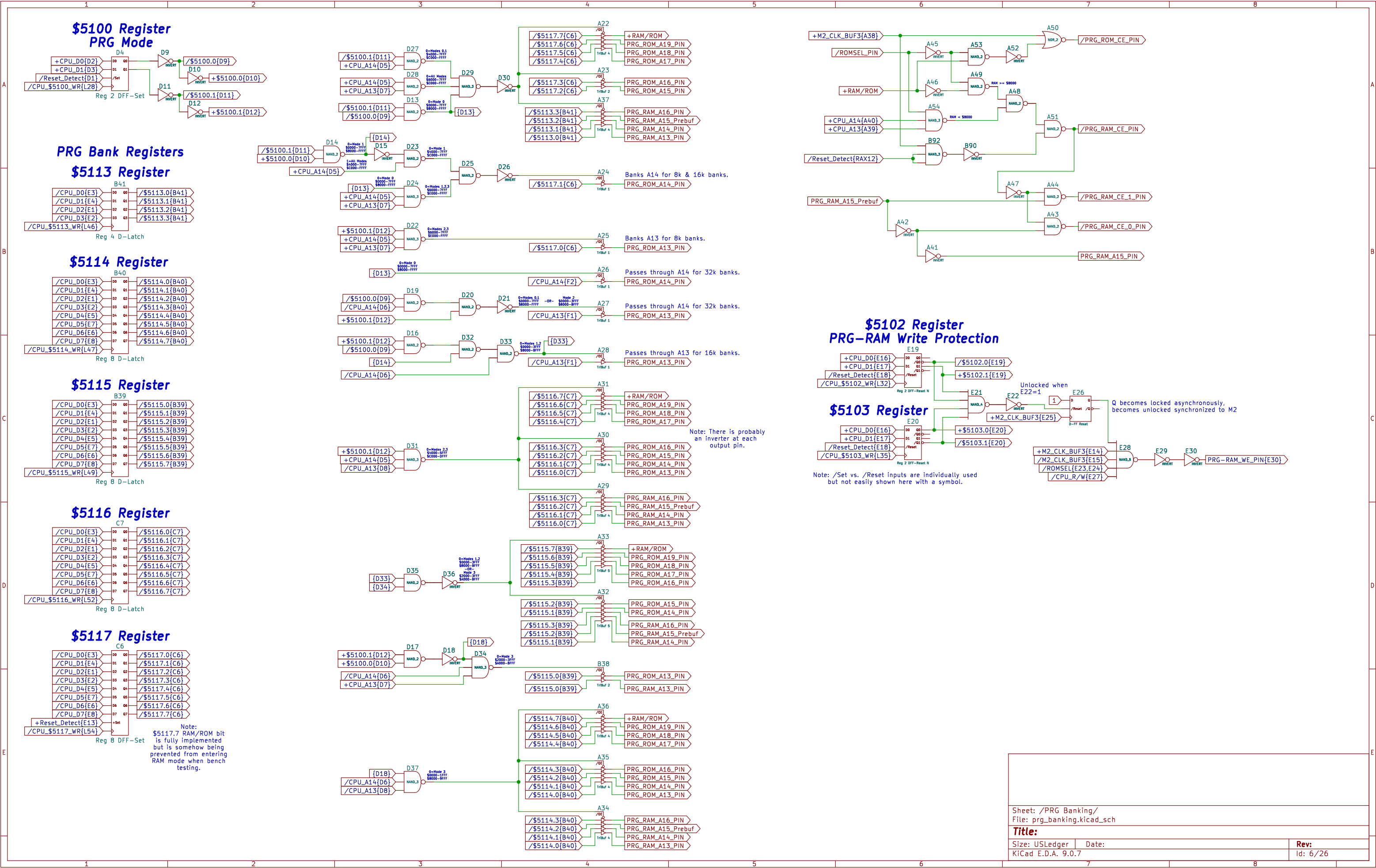
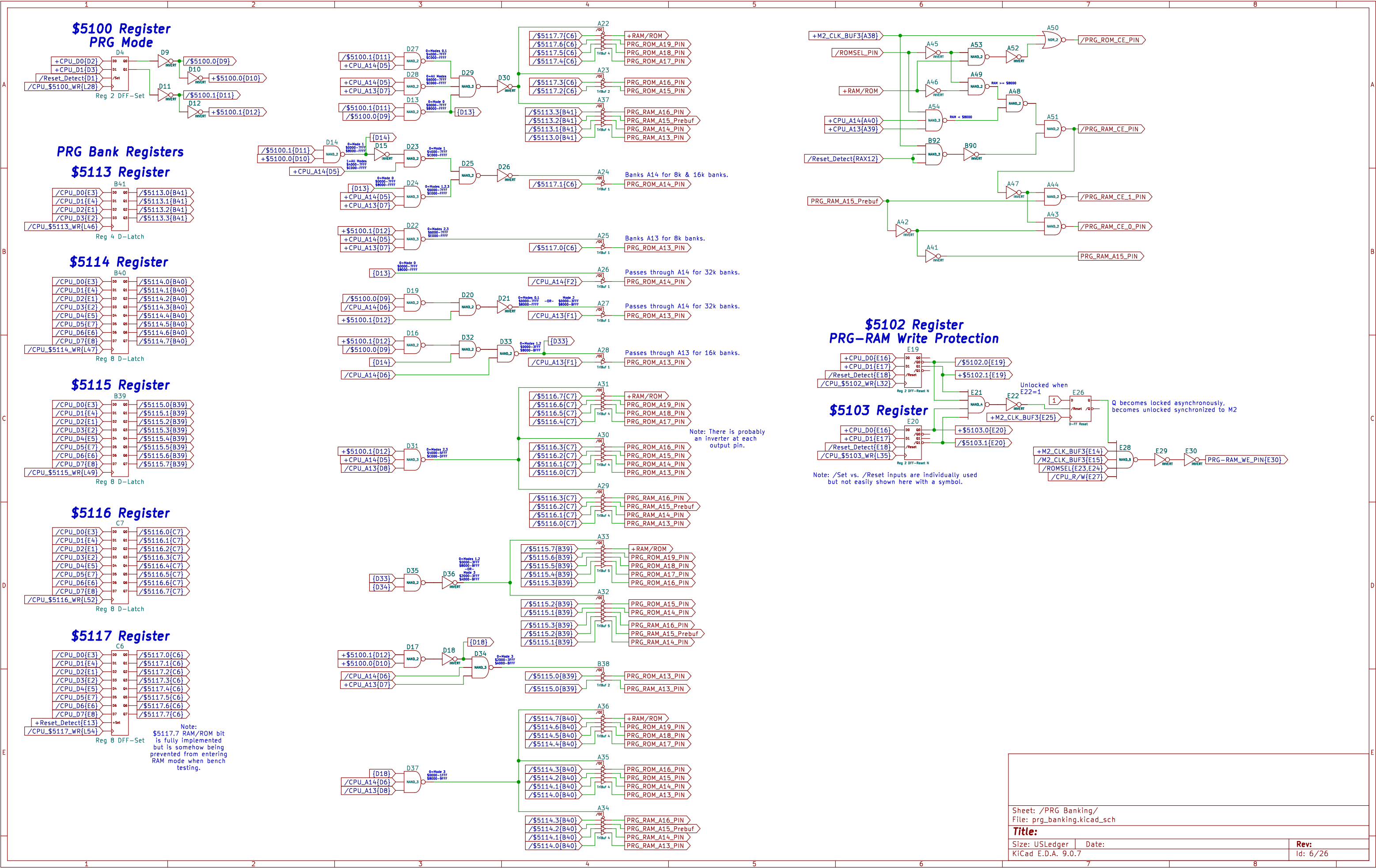
PPU Address Bus Buffering



Note:
/PPU_WR_PIN has an inverter
at the pin but does not go
to any buffers beyond that.





[illegible][illegible][illegible]

\$5100 Register PRG Mode

Reg 2 DFF-Set

PRG Bank Registers

\$5113 Register

Reg 4 D-Latch

\$5114 Register

Reg 8 D-Latch

\$5115 Register

Reg 8 D-Latch

\$5116 Register

Reg 8 D-Latch

\$5117 Register

Reg 8 DFF-Set

Note: \$5117.7 RAM/ROM bit is fully implemented but is somehow being prevented from entering RAM mode when bench testing.

\$5102 Register PRG-RAM Write Protection

Reg 2 DFF-Reset N

\$5103 Register

Reg 2 DFF-Reset N

Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

Q becomes locked asynchronously, becomes unlocked synchronized to M2

Sheet: /PRG Banking/
File: prg_banking.kicad_sch

Title:

Size: USLedger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 6/26

[illegible]

\$5100 Register PRG Mode

Reg 2 DFF-Set

PRG Bank Registers \$5113 Register

Reg 4 D-Latch

\$5114 Register

Reg 8 D-Latch

\$5115 Register

Reg 8 D-Latch

\$5116 Register

Reg 8 D-Latch

\$5117 Register

Reg 8 DFF-Set

Note: \$5117.7 RAM/ROM bit is fully implemented but is somehow being prevented from entering RAM mode when bench testing.

Banks A14 for 8k & 16k banks.

Banks A13 for 8k banks.

Passes through A14 for 32k banks.

Passes through A13 for 16k banks.

Note: There is probably an inverter at each output pin.

\$5102 Register PRG-RAM Write Protection

Reg 2 DFF-Reset N

\$5103 Register

Reg 2 DFF-Reset N

Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

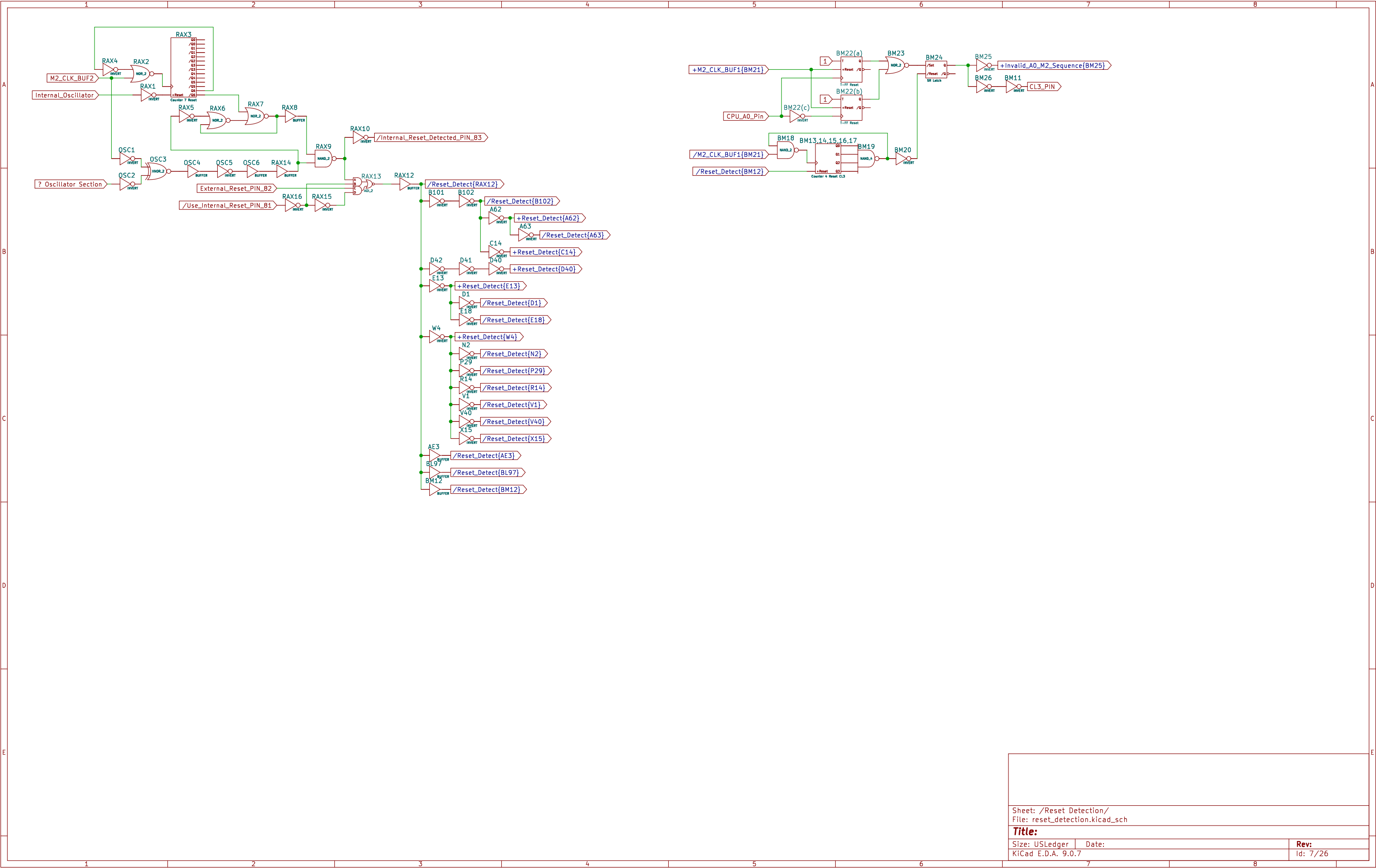
Unlocked when E22=1

Q becomes locked asynchronously, becomes unlocked synchronized to M2

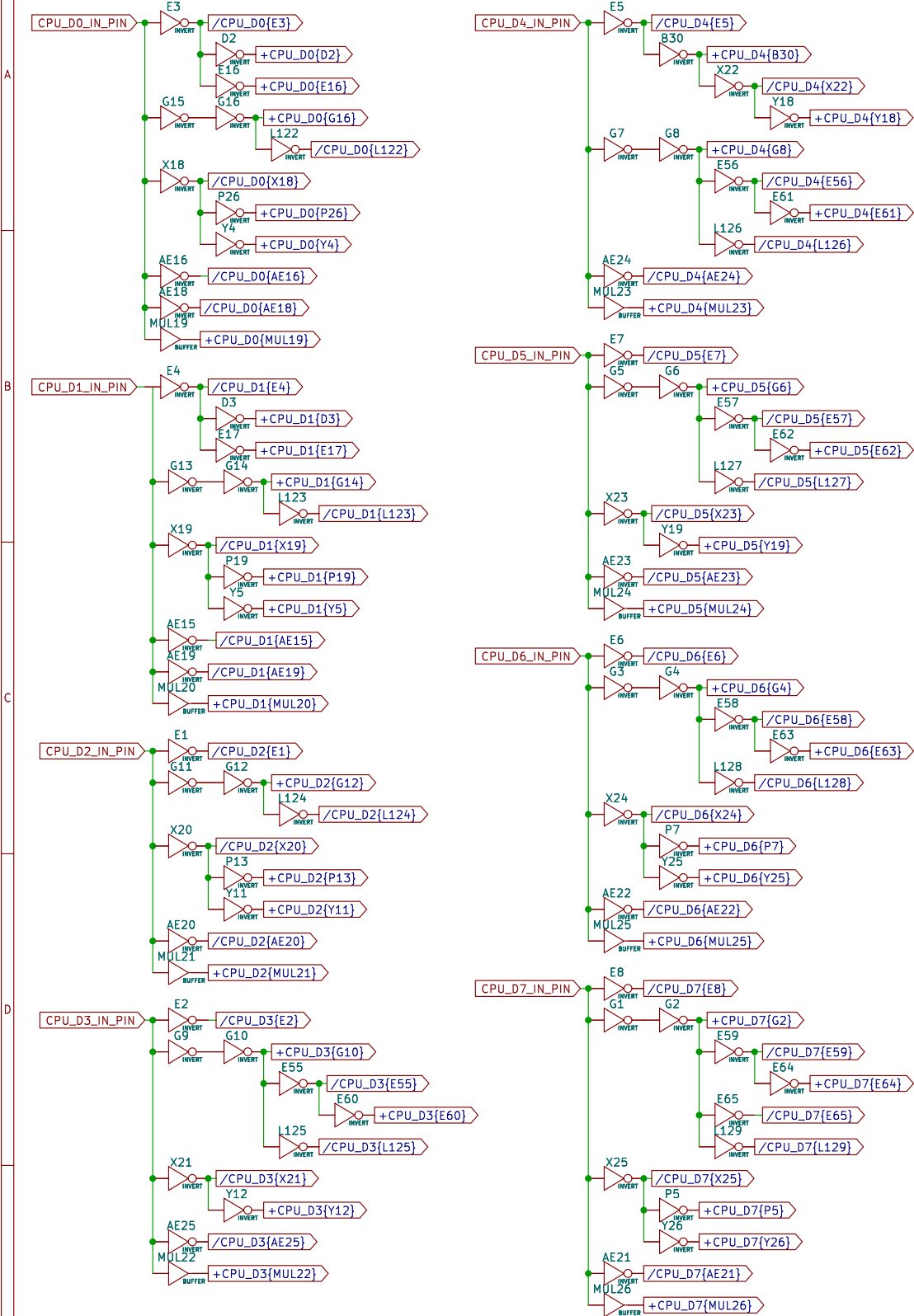
Sheet: /PRG Banking/
File: prg_banking.kicad_sch

Title:		Rev:
Size: USLdger	Date:	
KiCad E.D.A. 9.0.7		Id: 6/26

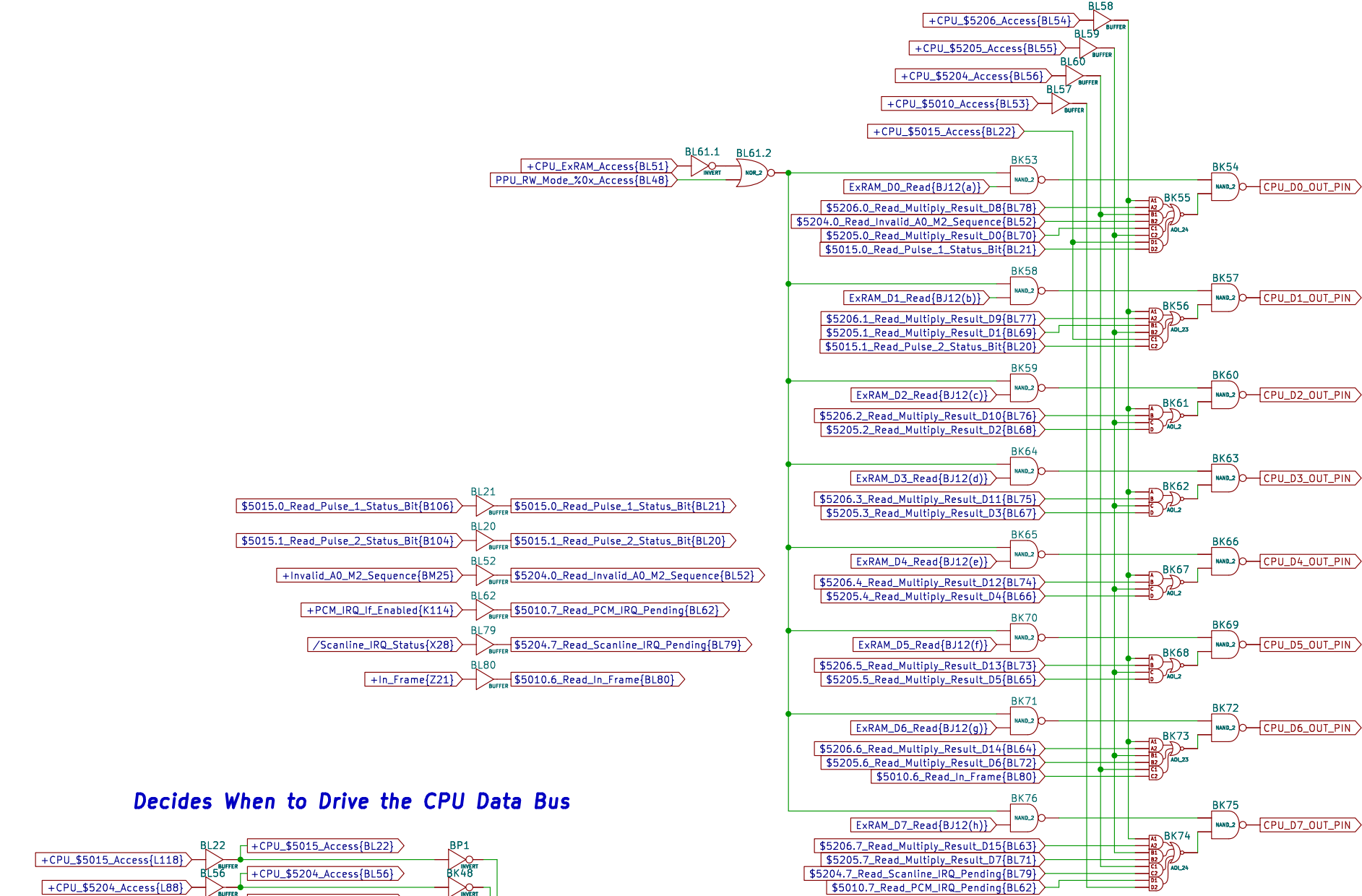
[illegible]



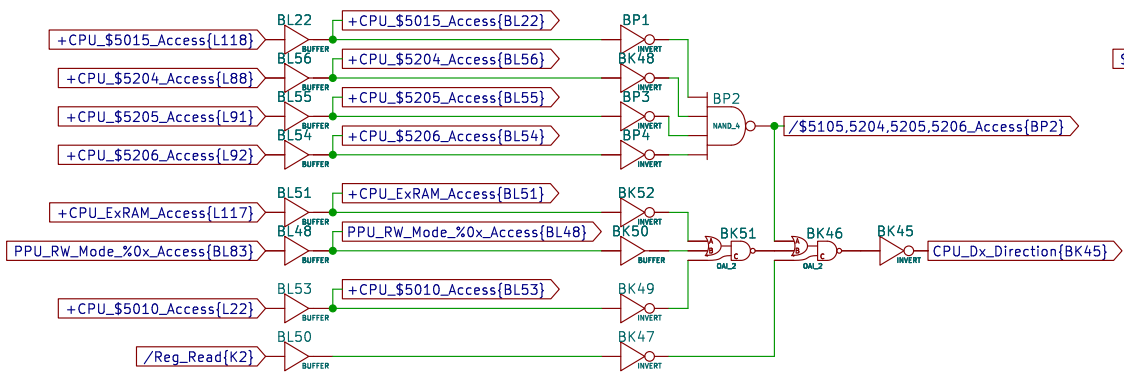
Incoming CPU Data Bus Buffering Bonanza



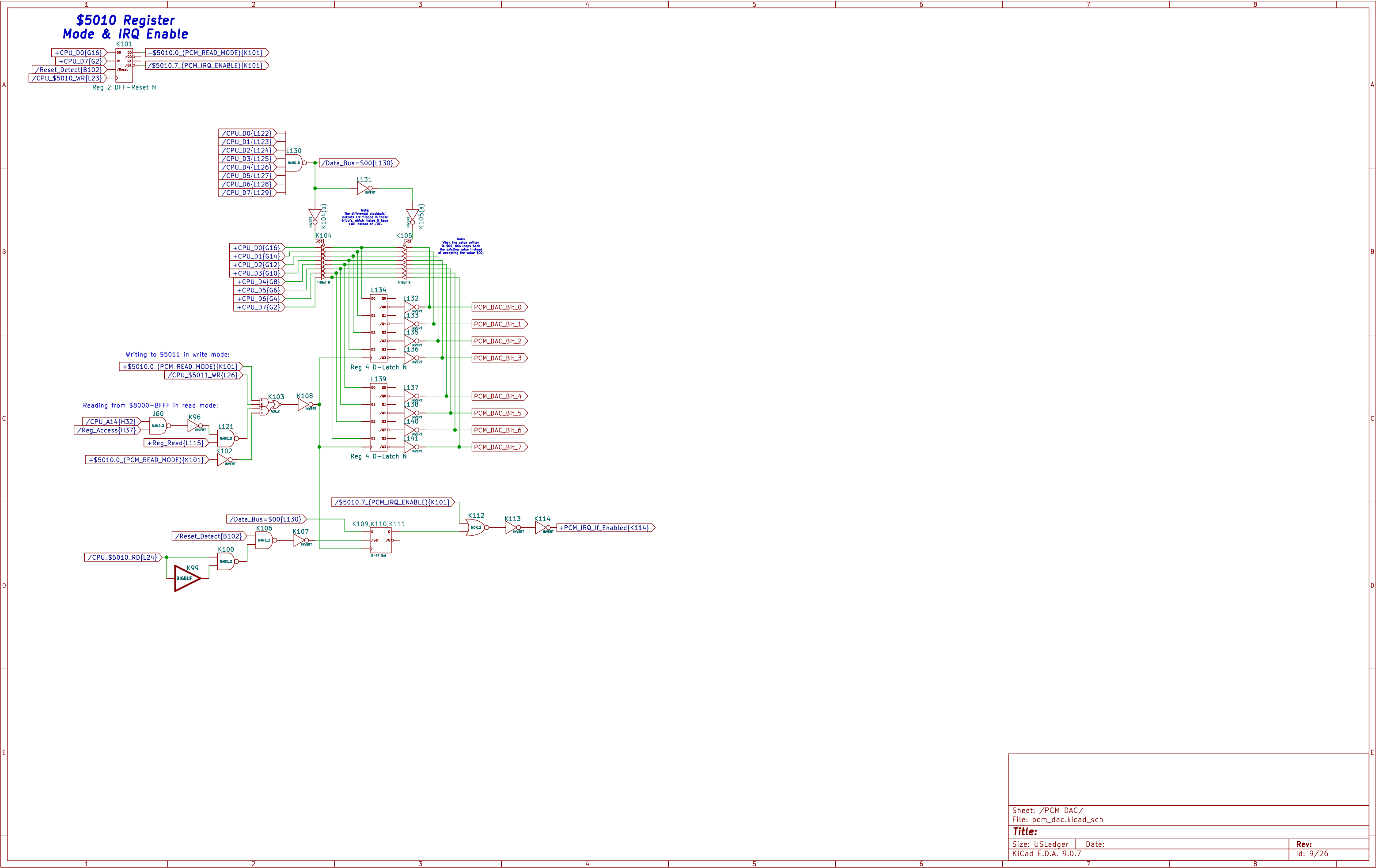
Multiplexing Data to be Driven onto the CPU Data Bus

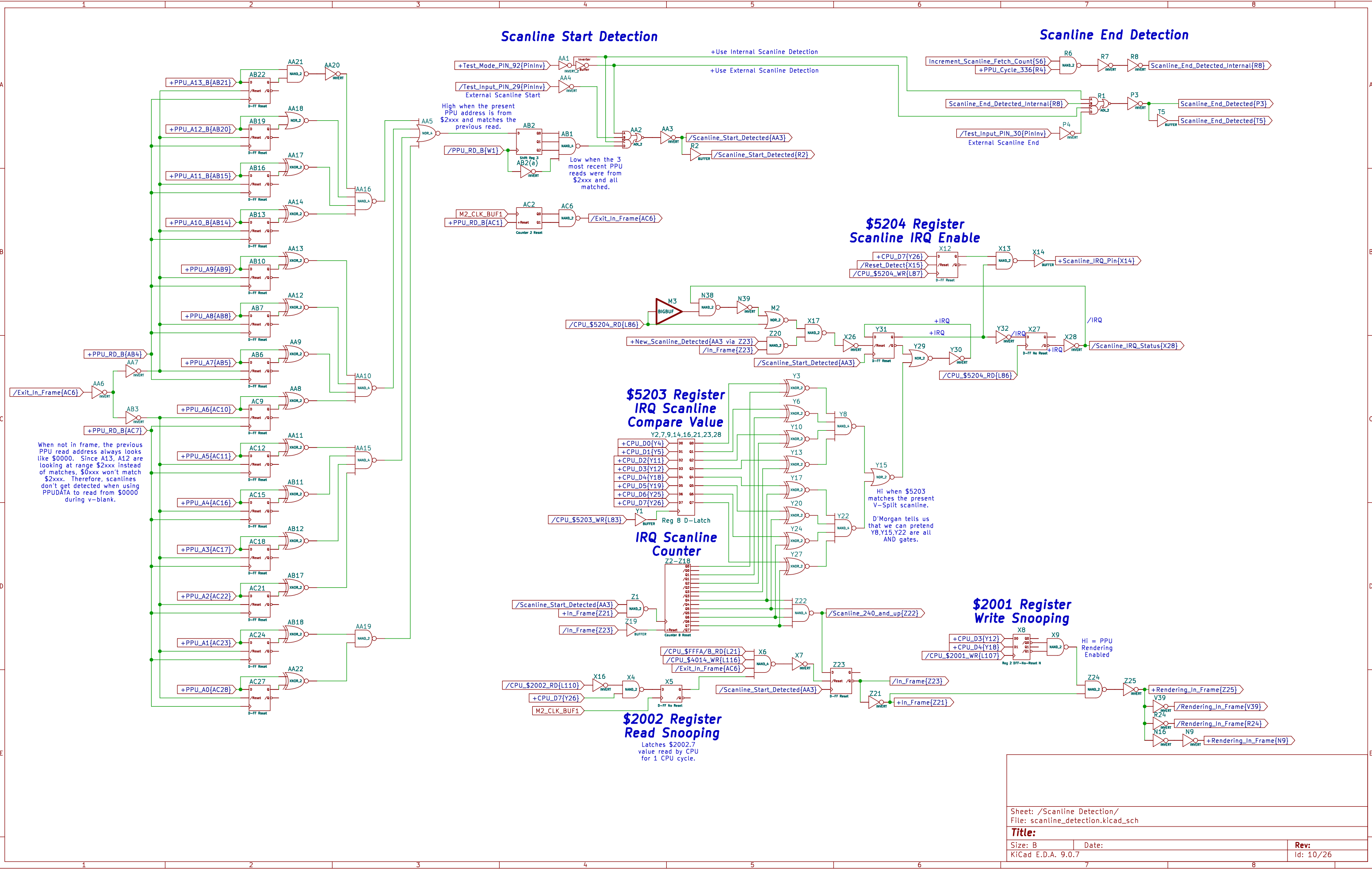


Decides When to Drive the CPU Data Bus



\$5010 Register Mode & IRQ Enable





When not in frame, the previous PPU read address always looks like \$0000. Since A13, A12 are looking at range \$2xxx instead of matches, \$0xxx won't match \$2xxx. Therefore, scanlines don't get detected when using PPUDATA to read from \$0000 during v-blank.

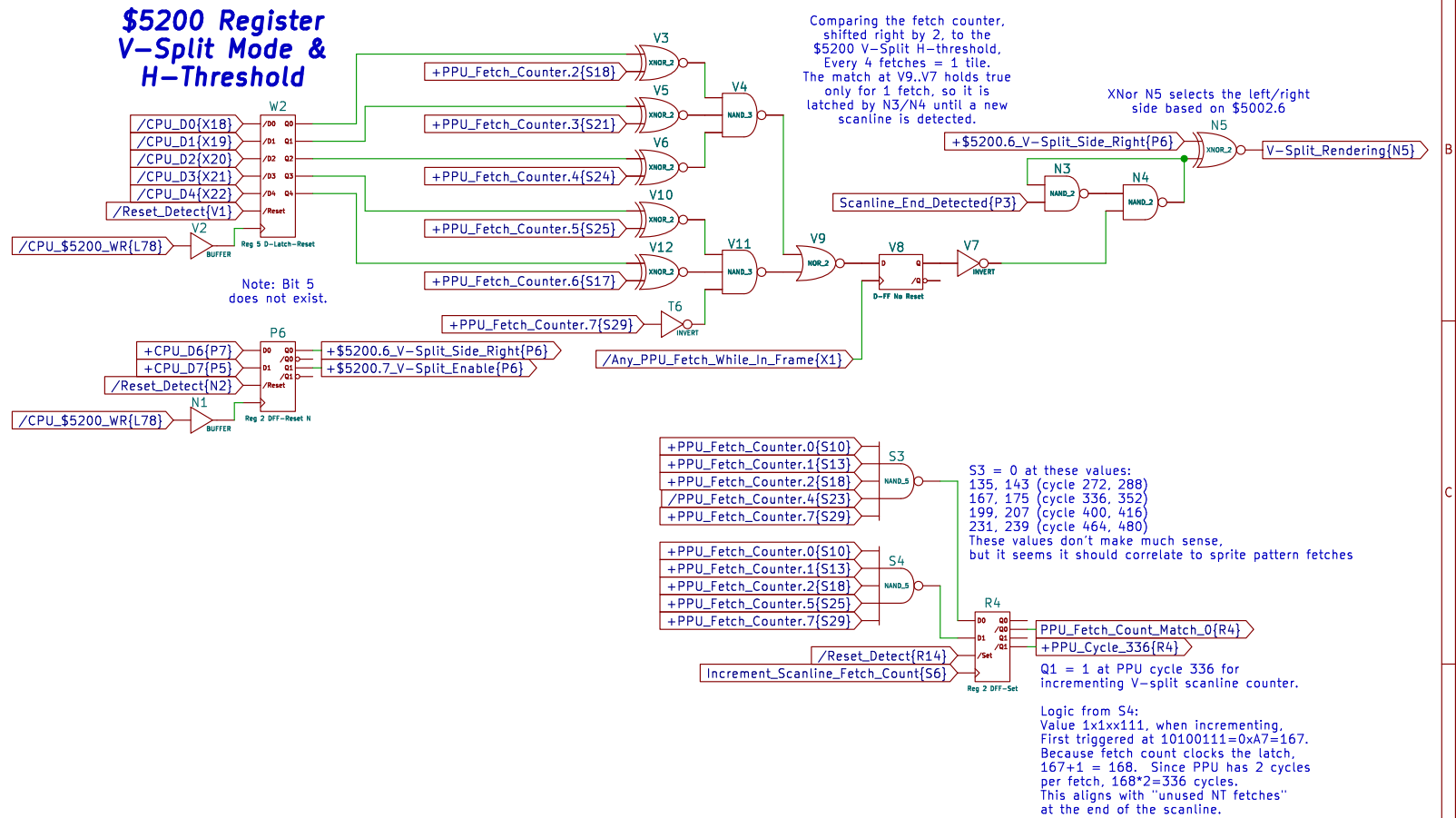
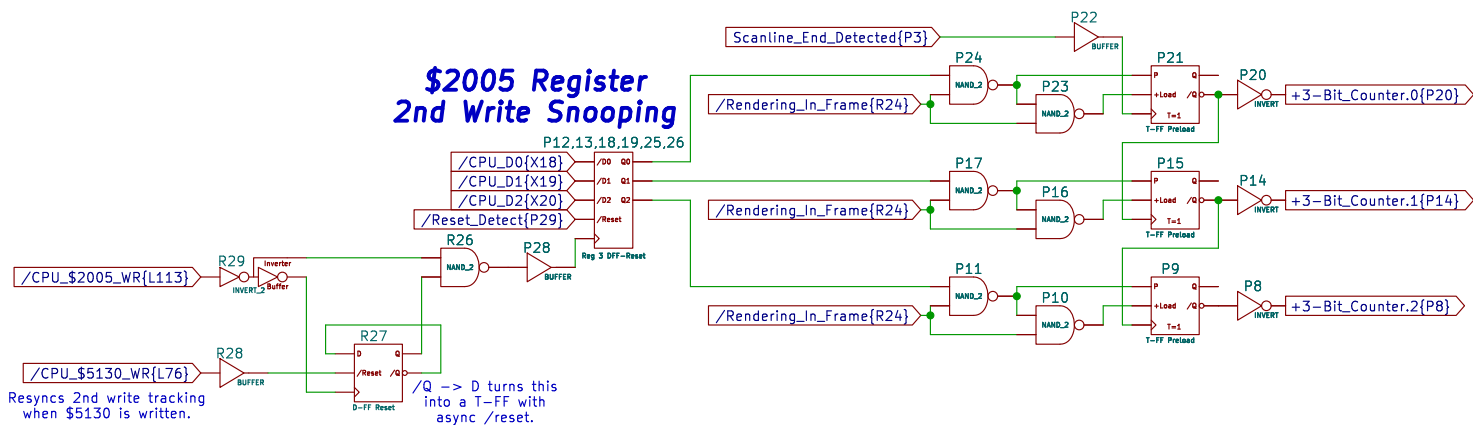
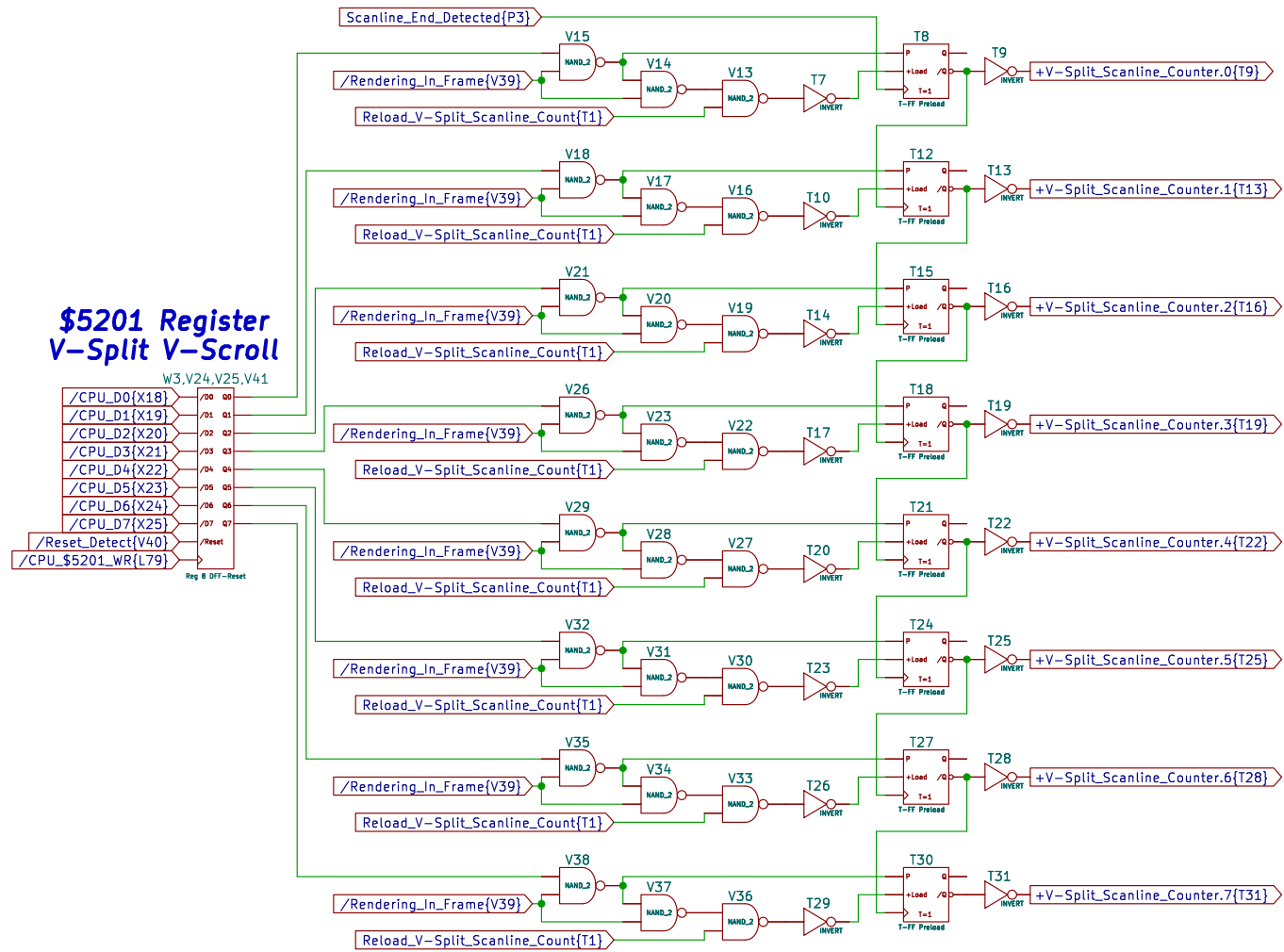
High when the present PPU address is from \$2xxx and matches the previous read.

Low when the 3 most recent PPU reads were from \$2xxx and all matched.

Hi when \$5203 matches the present V-Split scanline.

D'Morgan tells us that we can pretend Y8,Y15,Y22 are all AND gates.

Latches \$2002.7 value read by CPU for 1 CPU cycle.

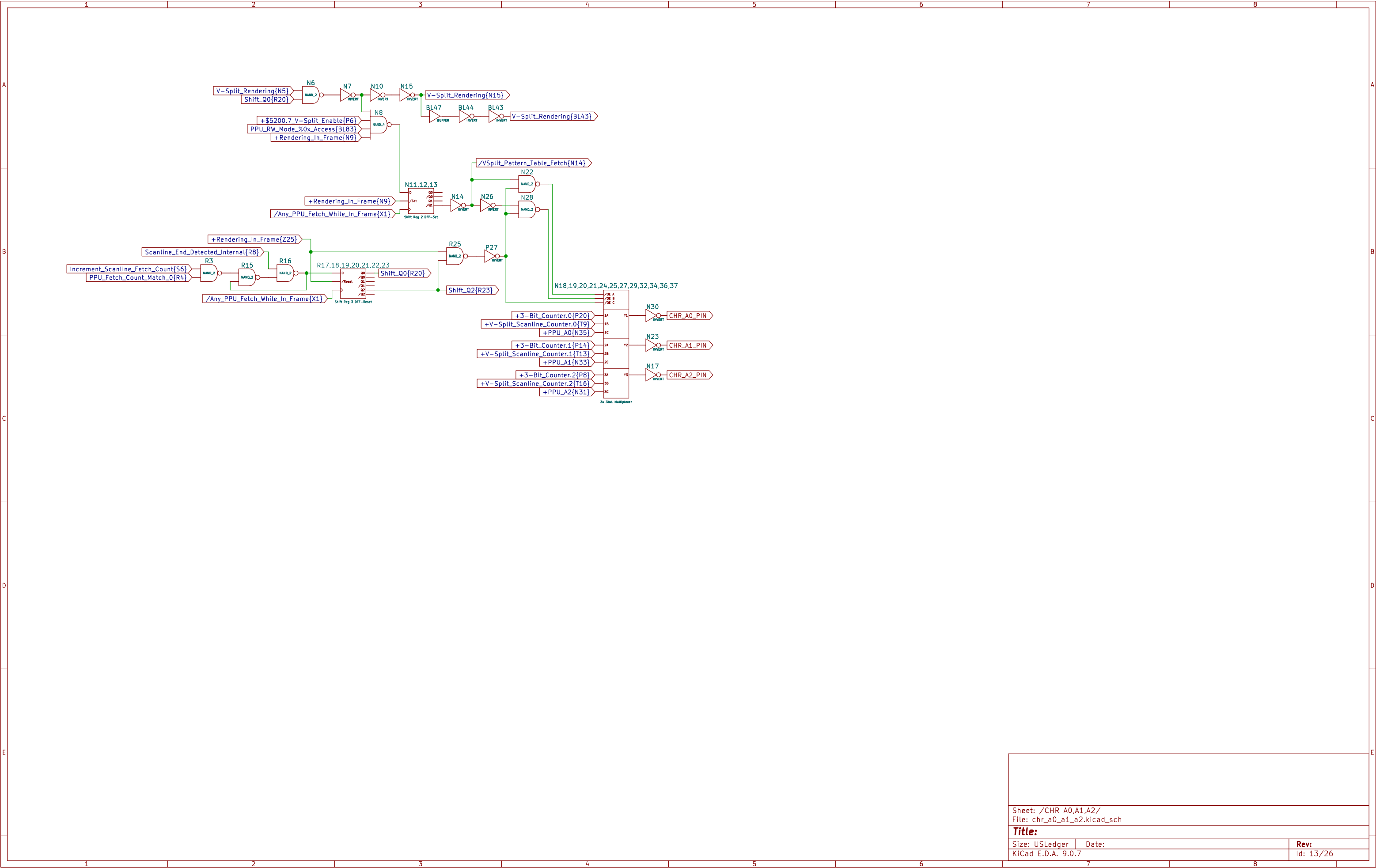


Id: 11/26

1	2	3	4	5	6	7	8	
---	---	---	---	---	---	---	---	--

Input Channel	Input Address Range	Output Channel	Output Address Range
+PPU_Fetch_Counter.0{S10}	A/B	PPU_Fetch_Mux_0{M1}	1A
+PPU_Fetch_Counter.2{S18}	1A	PPU_Fetch_Mux_1{M1}	1B
+PPU_Fetch_Counter.4{S24}	1B	PPU_Fetch_Mux_2{M1}	2A
+PPU_Fetch_Counter.3{S21}	2A	PPU_Fetch_Mux_3{M1}	2B
+PPU_Fetch_Counter.5{S25}	2B	PPU_Fetch_Mux_4{M1}	3A
+PPU_Fetch_Counter.4{S24}	3A	PPU_Fetch_Mux_5{M1}	3B
+PPU_Fetch_Counter.6{S17}	3B	PPU_Fetch_Mux_6{M1}	4A
+PPU_Fetch_Counter.5{S25}	4A	PPU_Fetch_Mux_7{M1}	4B
+V-Split_Scanline_Counter.5{T25}	4B	PPU_Fetch_Mux_8{M1}	5A
+PPU_Fetch_Counter.6{S17}	5A	PPU_Fetch_Mux_9{M1}	5B
+V-Split_Scanline_Counter.6{T28}	5B		
+V-Split_Scanline_Counter.3{T19}	6A		
+V-Split_Scanline_Counter.7{T31}	6B		
+V-Split_Scanline_Counter.4{T22}	7A		
	7B		
+V-Split_Scanline_Counter.5{T25}	8A		
	8B		
+V-Split_Scanline_Counter.6{T28}	9A		
	9B		
+V-Split_Scanline_Counter.7{T31}	10A		
	10B		

10x 2nd Multiplexer



\$5104 Register ExRAM Mode

Consider: ExRAM NEVER contains pattern tables and that should be related to how PPU /A13 is being used here.

Question: Why should we care about PPU /A13 AT ALL when the CPU has control of ExRAM address bus?

Consider: ExRAM can still be used as a nametable when ExRAM mode is set to CPU mode.

TRUE when: ExRAM is mode \$1x, which gives CPU access but no PPU access, *OR* PPU is accessing NT or AT

That doesn't make sense because what if you have a nametable assigned to ExRAM in \$5105? It needs ExRAM Ax coming from PPU.

Signal A9 (+ExRAM Nametable Access) is VERY closely but no obvious connection

Writing to an ExRAM nametable via PPU via \$2000

Writing to ExRAM via \$5C00-5FFF in mode %10

This is the write signal for the ExRAM itself.

Sheet: /ExRAM Mode/
File: exram_mode.kicad_sch

Title:

Size: USLdger Date: Rev:

KiCad E.D.A. 9.0.7 Id: 14/26

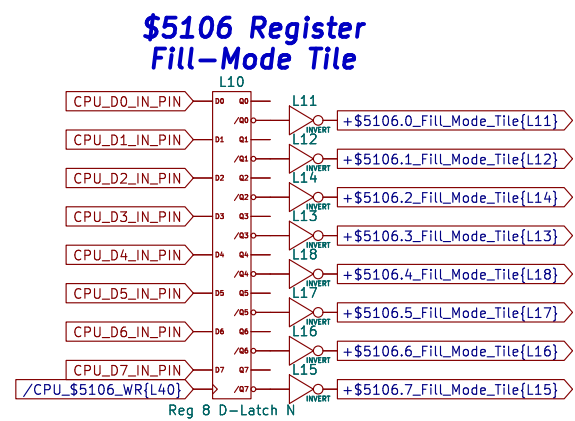
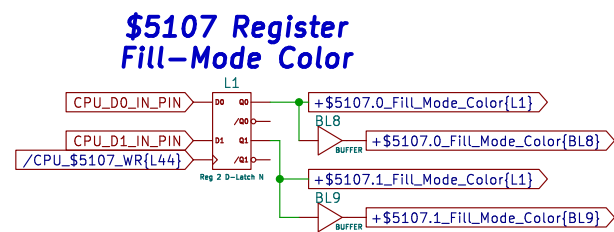
Consider:
ExRAM can still be used as a nametable
when ExRAM mode is set to CPU mode.

That doesn't make sense because
what if you have a nametable
assigned to ExRAM in \$5105?
It needs ExRAM A_x coming from P

Sheet: /ExRAM Mode/
 File: exram_mode.kicad_sch

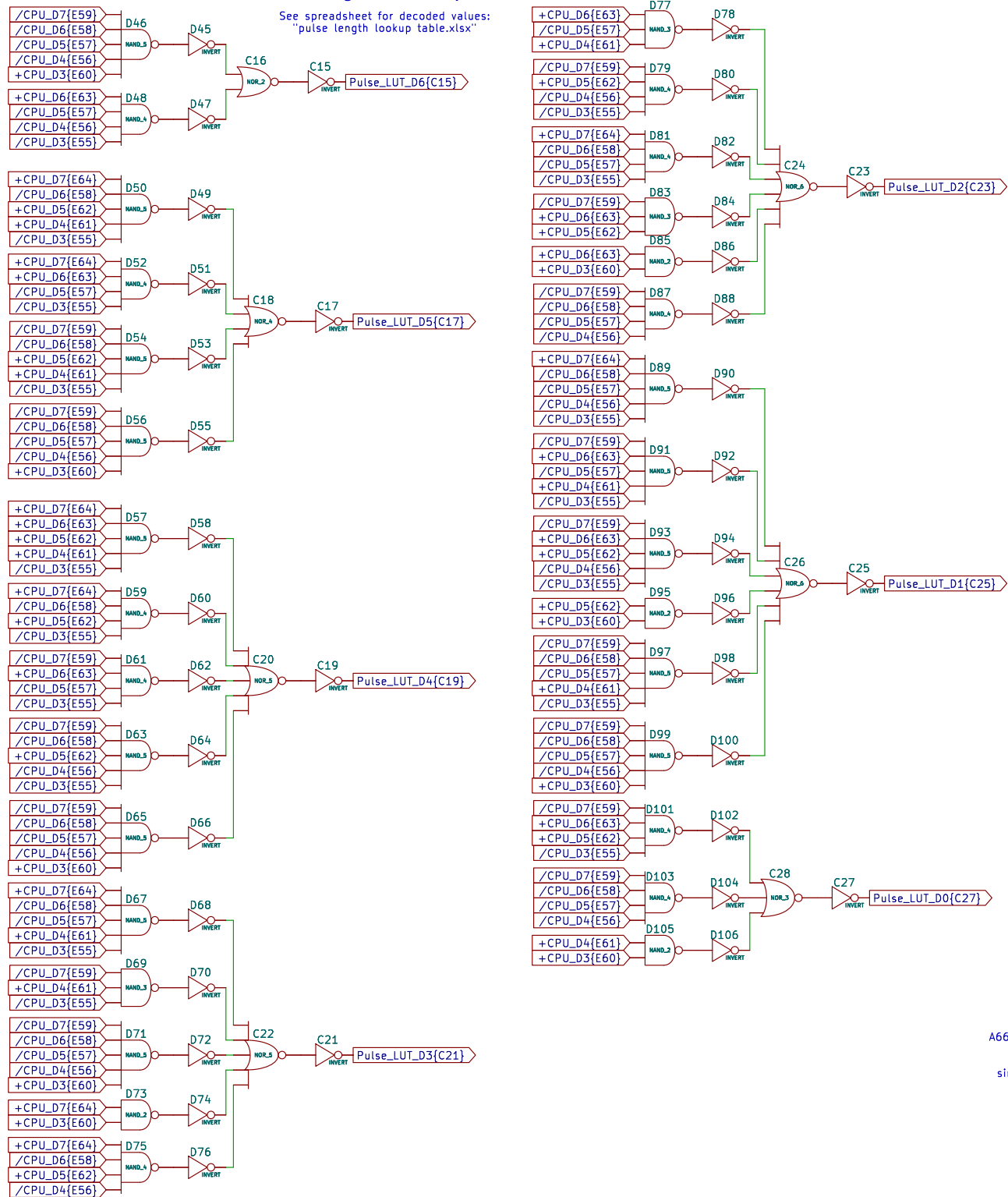
Title:

Size: USLedge	Date:	Rev:
KiCad E.D.A. 9.0.7		Id: 14/26



5-Bit to 7-Bit Pulse Length Lookup Table

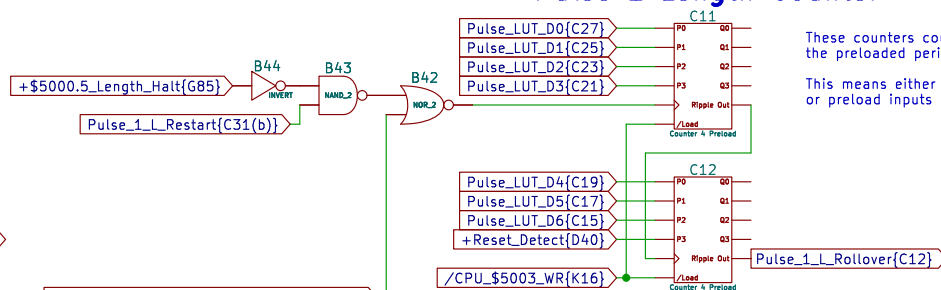
See spreadsheet for decoded values:
"pulse length lookup table.xlsx"



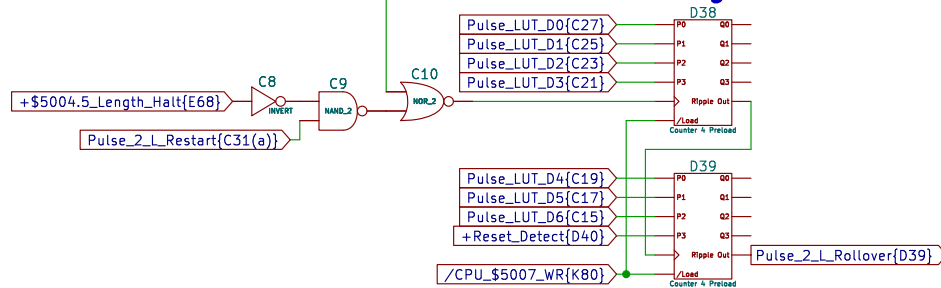
\$5003 Register Pulse 1 Length Counter

These counters count "down"
the preloaded period value.

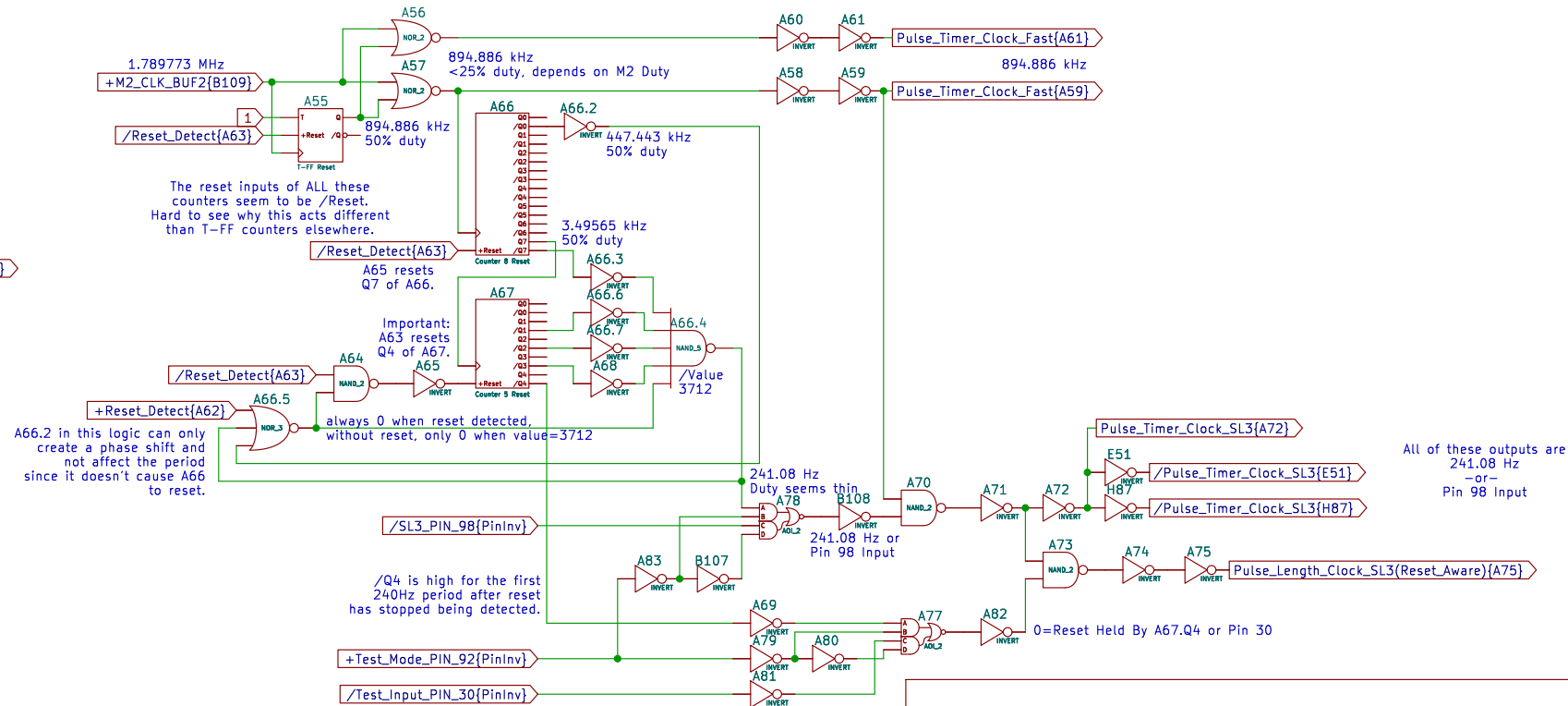
This means either the outputs
or preload inputs are inverted.



\$5007 Register Pulse 2 Length Counter



Pulse Channel Clock Source



Sheet: /Pulse Length Lookup Table/
File: pulse_length_lookup_table.kicad_sch

Title:

Size: USLdger Date:

Rev:

KiCad E.D.A. 9.0.7

Id: 16/26

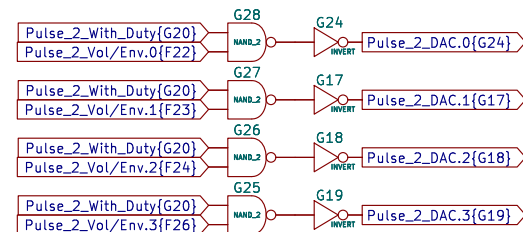
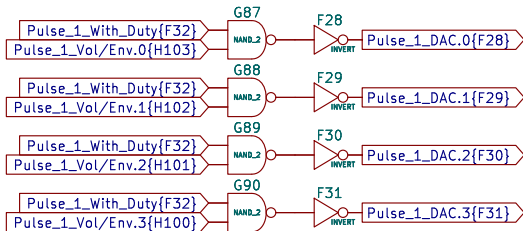
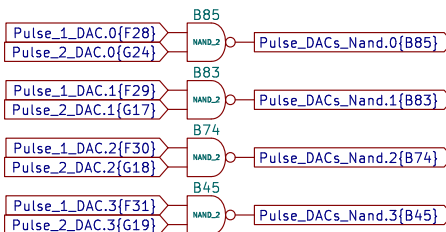


Figure 10 illustrates the block diagram of the Pulse DACs. It shows four parallel processing paths, each taking two 16-bit inputs and producing a 16-bit output. The inputs are labeled as Pulse_1_DAC and Pulse_2_DAC, and the outputs are labeled as Pulse_DACs_Nor. The bit ranges for the inputs and outputs are specified in the diagram.

- Path 1:** Pulse_1_DAC.0[F28] and Pulse_2_DAC.0[G24] are inputs to a NOR gate (NOR_2), resulting in Pulse_DACs_Nor.0[B84].
- Path 2:** Pulse_1_DAC.1[F29] and Pulse_2_DAC.1[G17] are inputs to a NOR gate (NOR_2), resulting in Pulse_DACs_Nor.1[B82].
- Path 3:** Pulse_1_DAC.2[F30] and Pulse_2_DAC.2[G18] are inputs to a NOR gate (NOR_2), resulting in Pulse_DACs_Nor.2[B73].
- Path 4:** Pulse_1_DAC.3[F31] and Pulse_2_DAC.3[G19] are inputs to a NOR gate (NOR_2), resulting in Pulse_DACs_Nor.3[B53].



Note: The adder's logic is verified correct in Pulse DAC Adder Logic.xlsx

This logic amounts to XOR

Pulse_DACs_Nand.0{B85}

Pulse_DACs_Nor.0{B84}

B50 INVERT

B49 NAND.2

B51 INVERT

Pulse_DACs_Nand.1{B83}

Pulse_DACs_Nor.1{B82}

B88 INVERT

B78 NAND.2

B79 INVERT

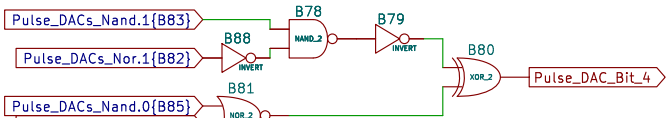
B80 XOR.2

Pulse_DAC_Bit_1

Pulse_DAC_Bit_2

Pulse_DAC_Bit_3

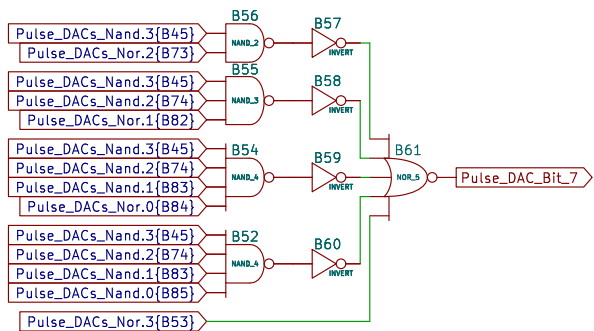
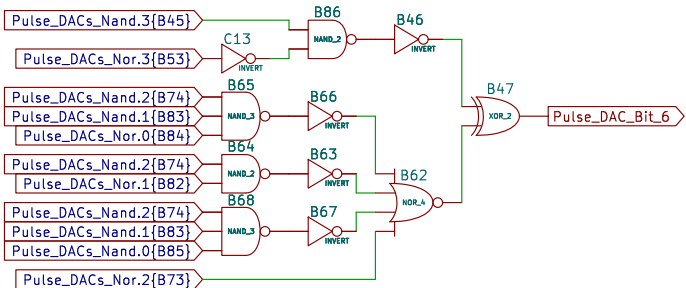
Pulse_DAC_Bit_4

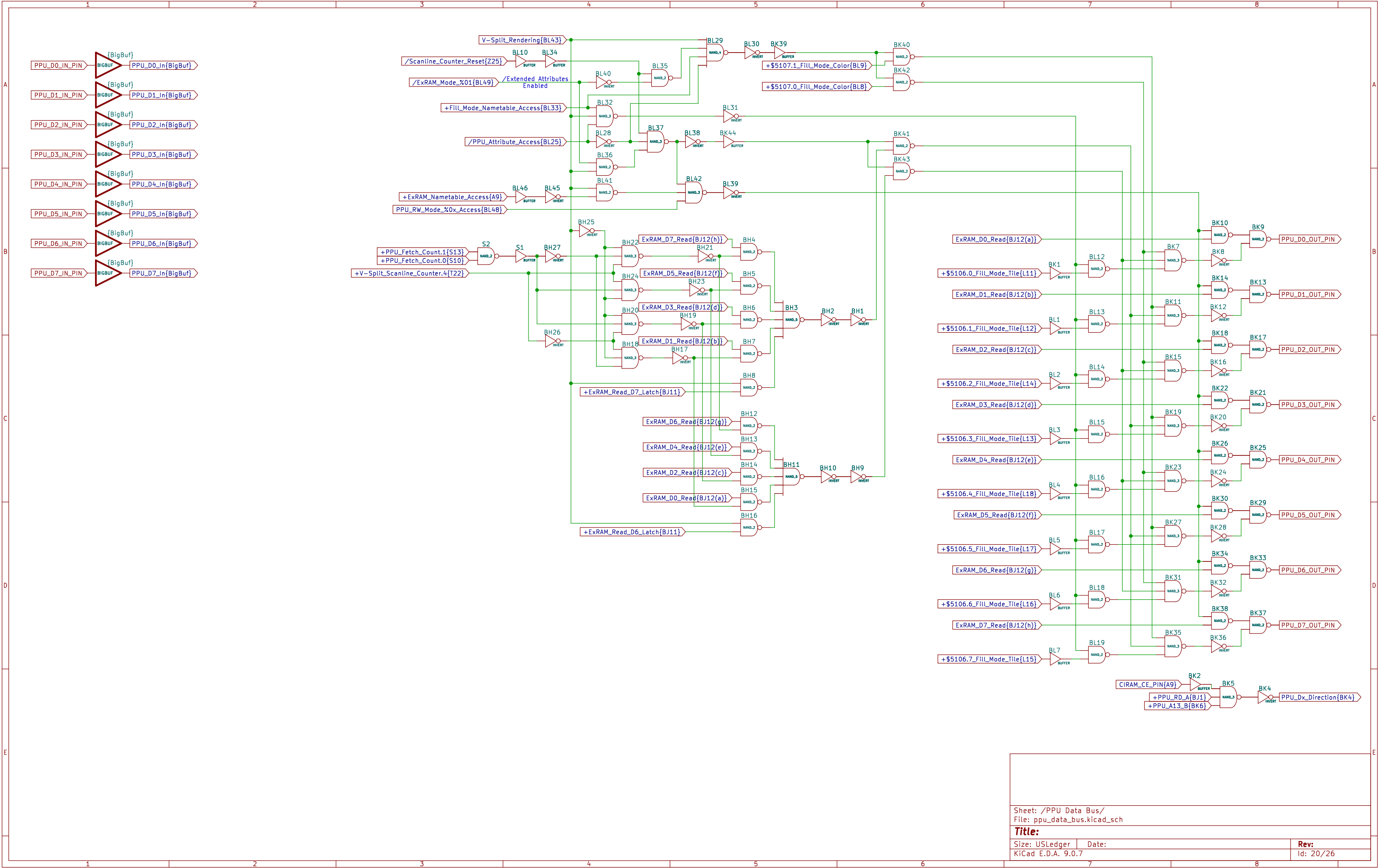


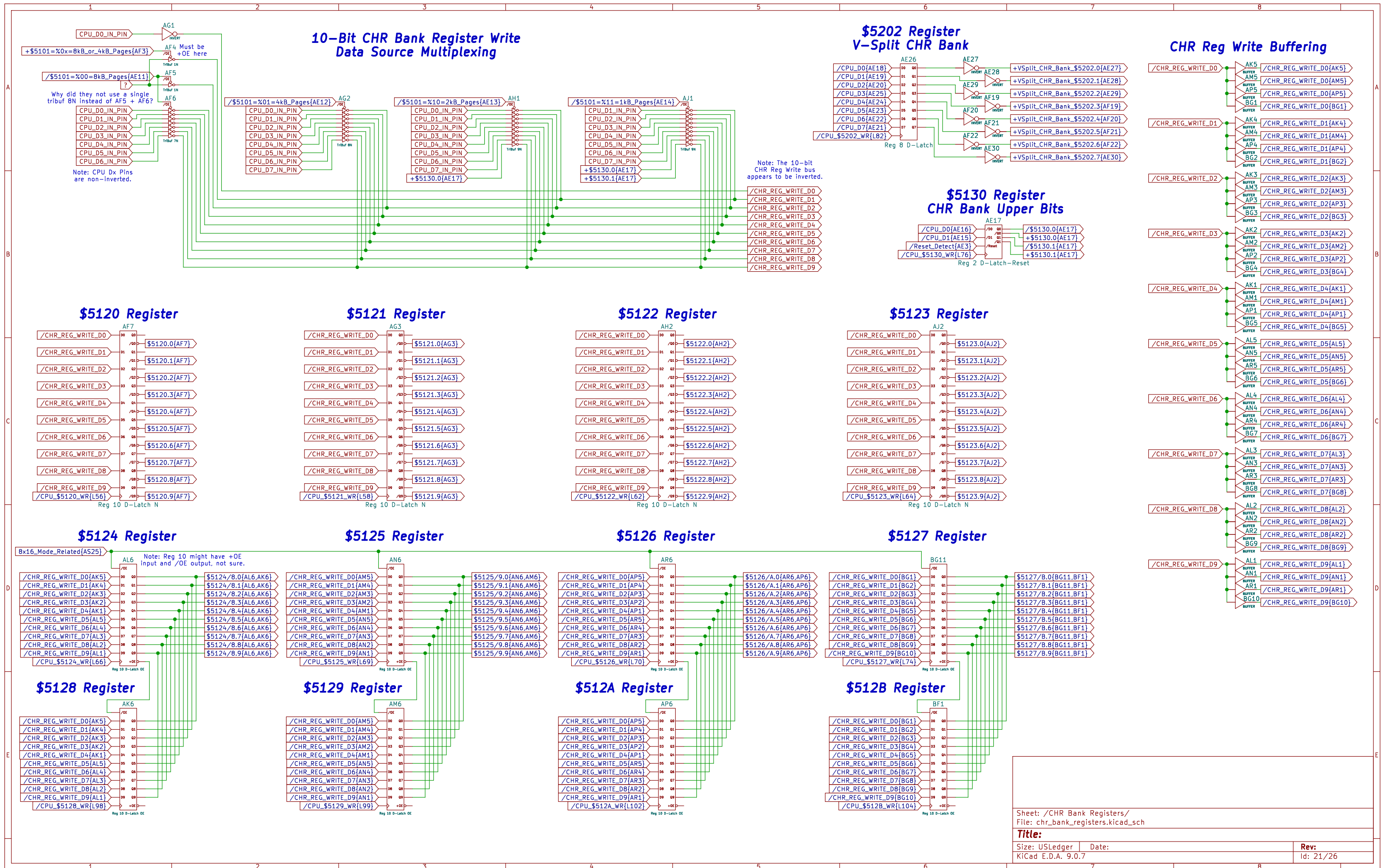
Lowest 3 bits
are not used.

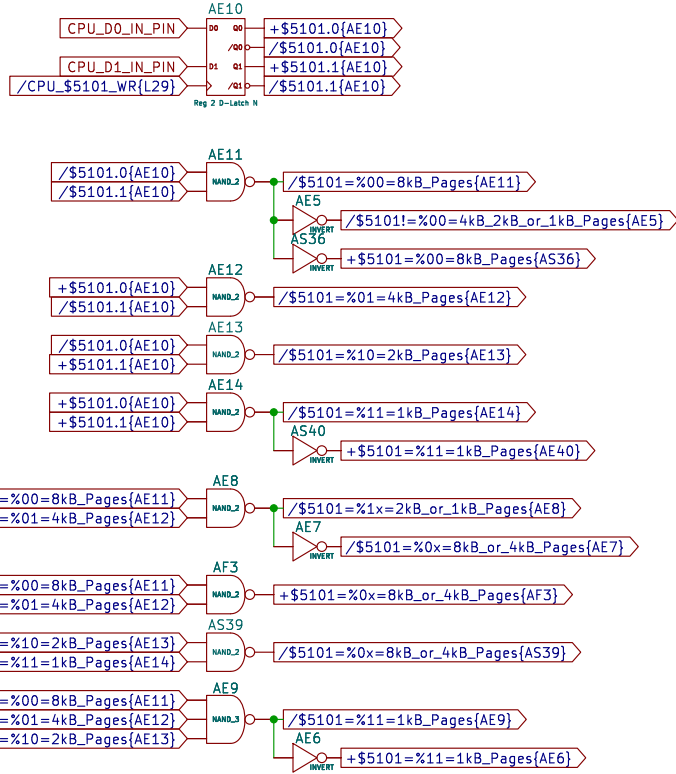
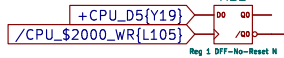
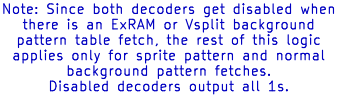
This

B80,B72,B47:
Though they look like
XNOR gates from
elsewhere, logically
these must be XOR.

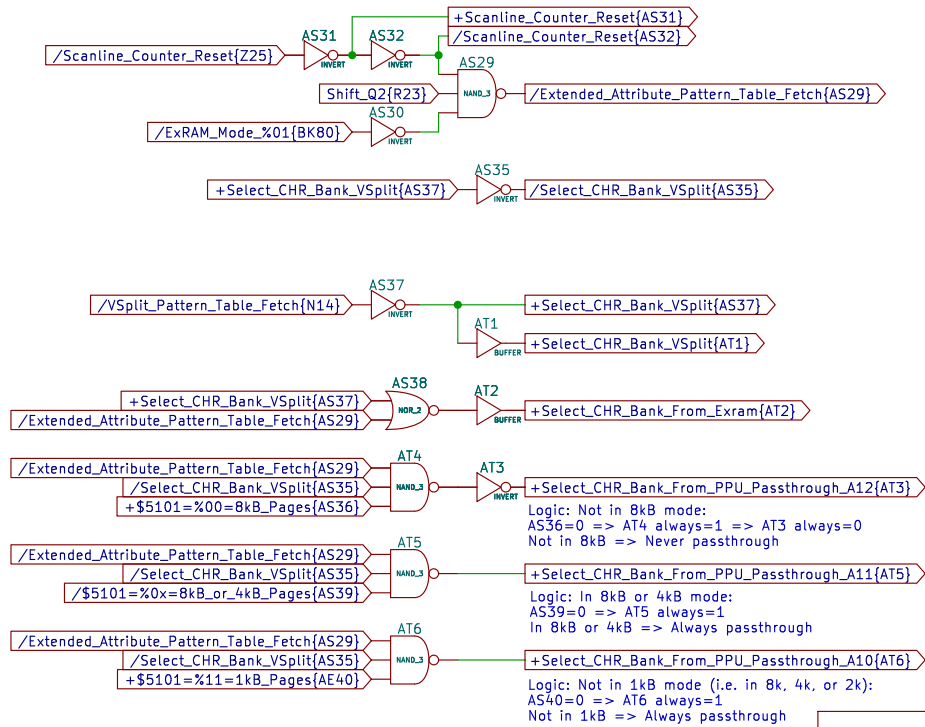




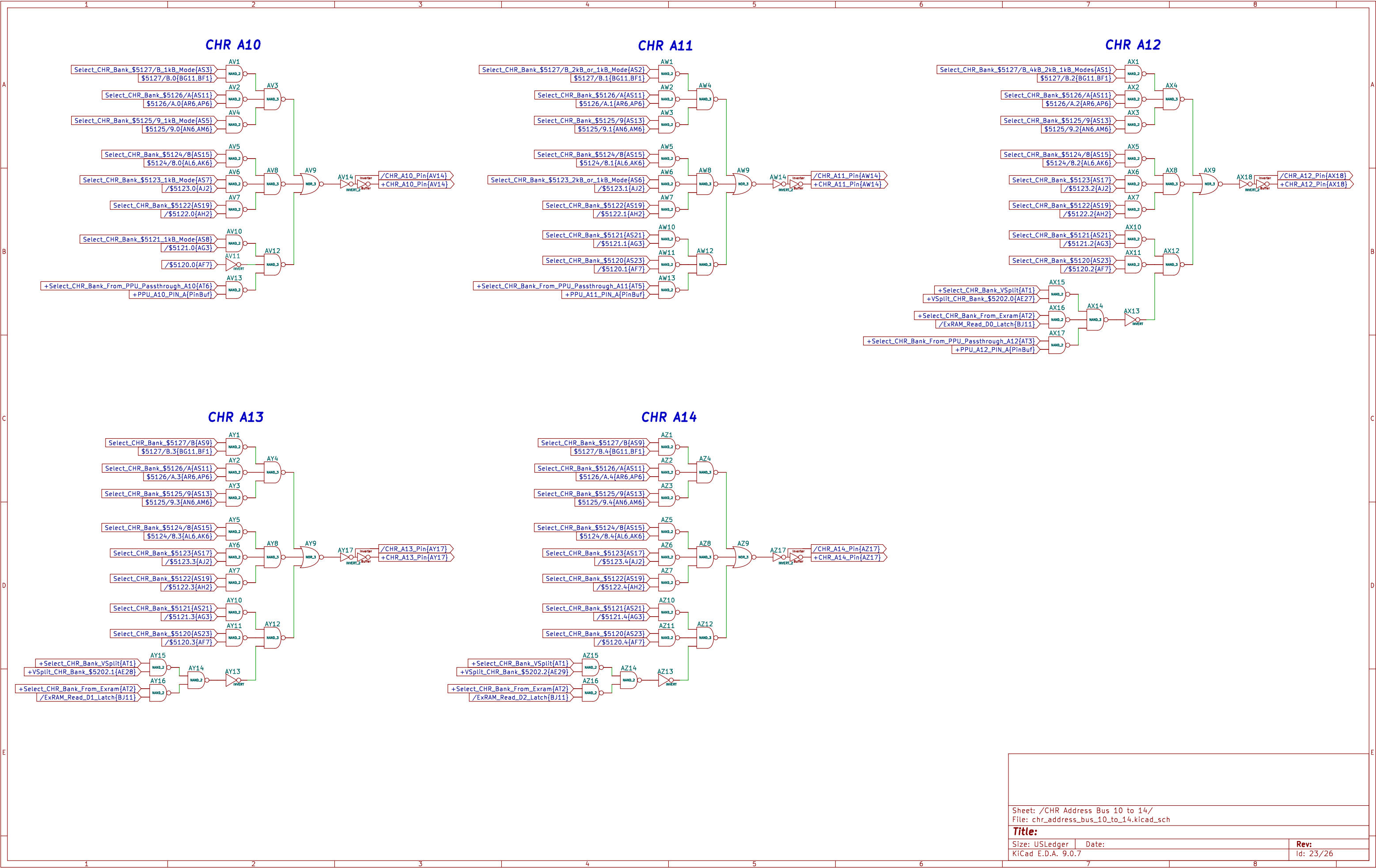


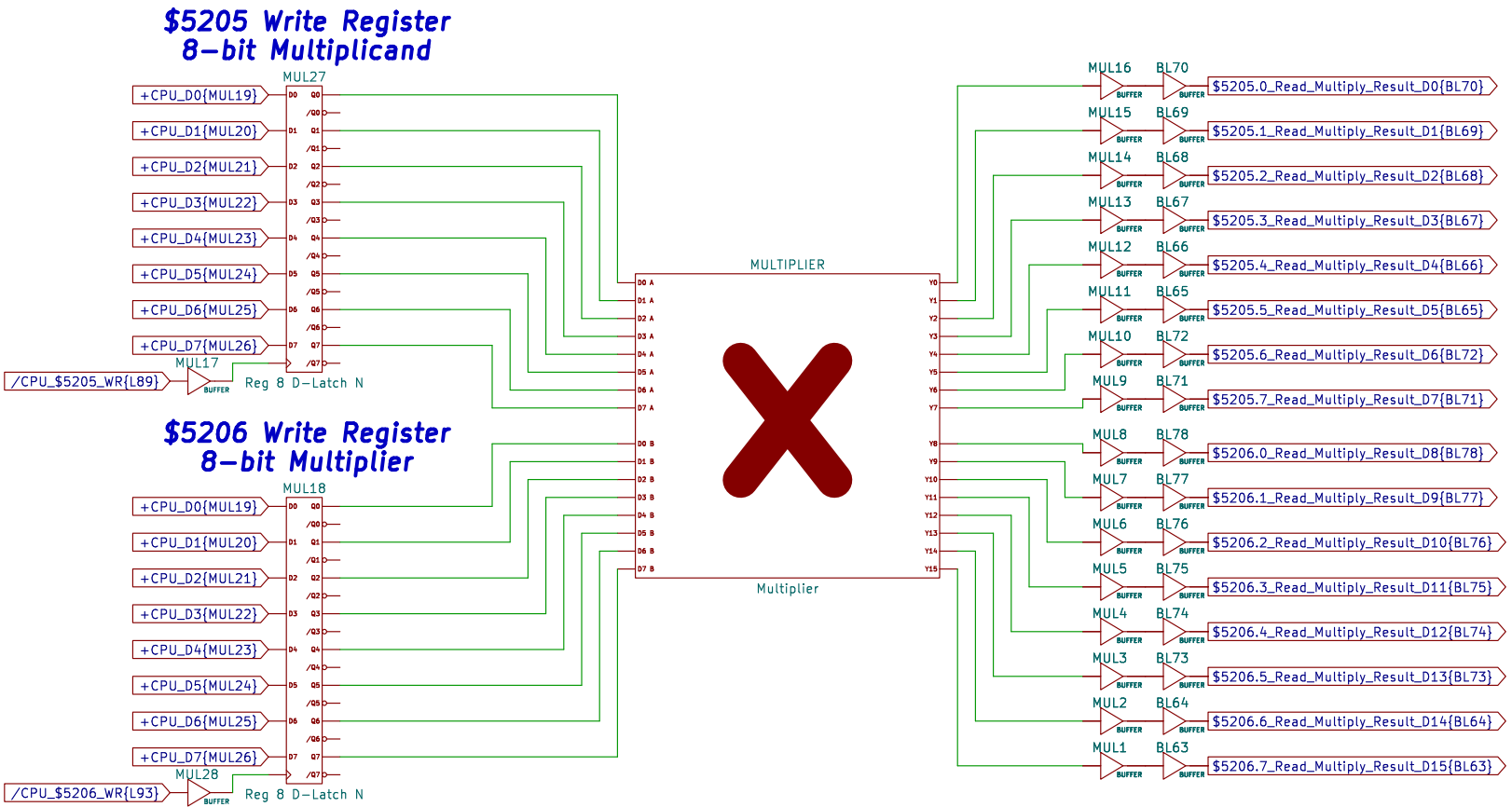


The logic in this area is
strangely unoptimized.



Size: USLedger	Date:	Rev:
KiCad E.D.A. 9.0.7		Id: 22/26





Sheet: /Multiplier/
File: multiplier.kicad_sch

Title:

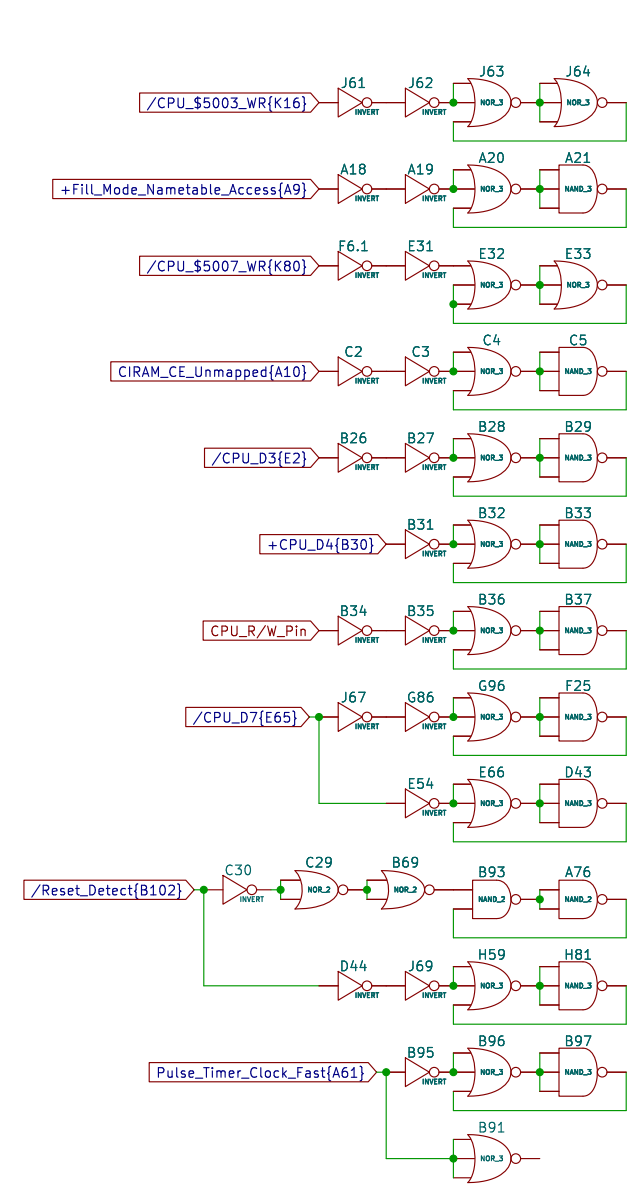
Size: USLedge
KiCad E.D.A. 9.0.7

Date:

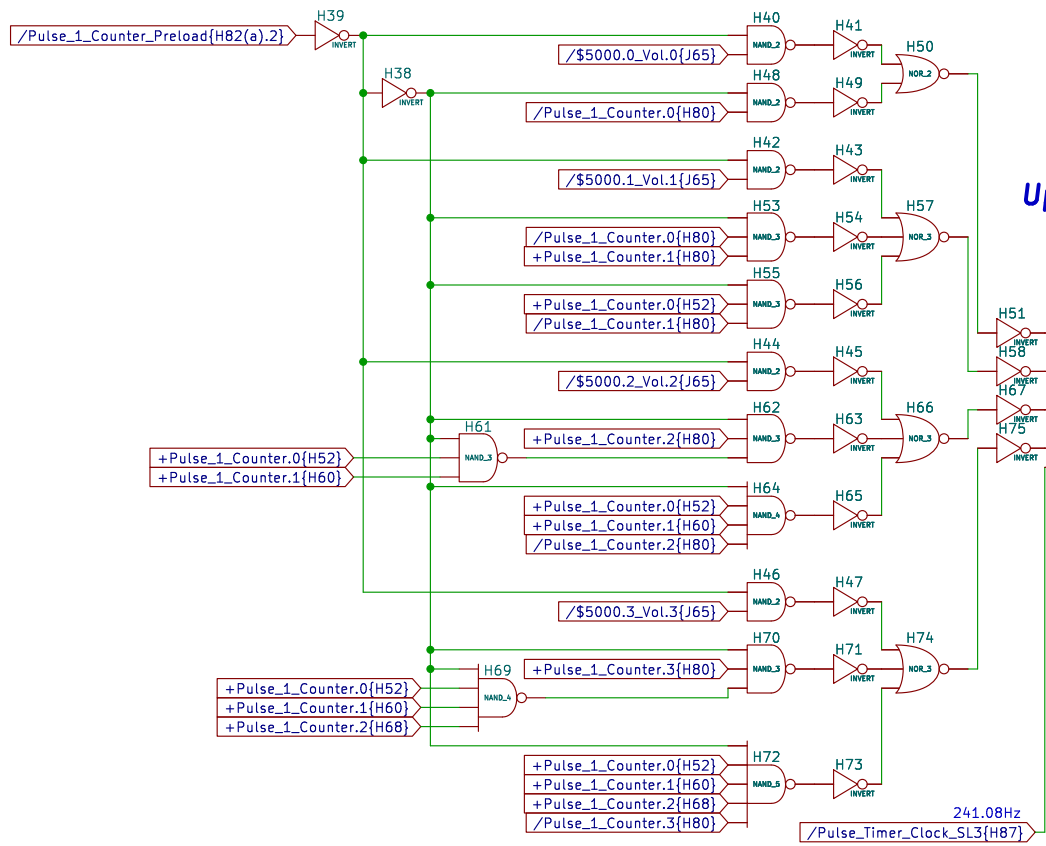
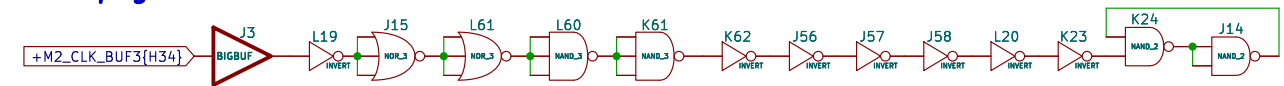
Rev:

Id: 25/26

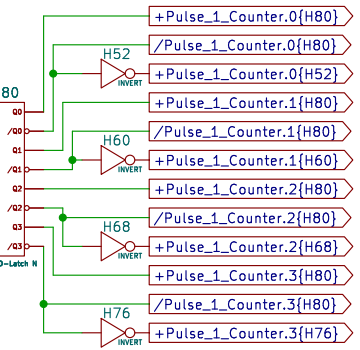
This sheet contains all residual items that don't serve any apparent purpose.



M2 Spaghetti That Goes Nowhere

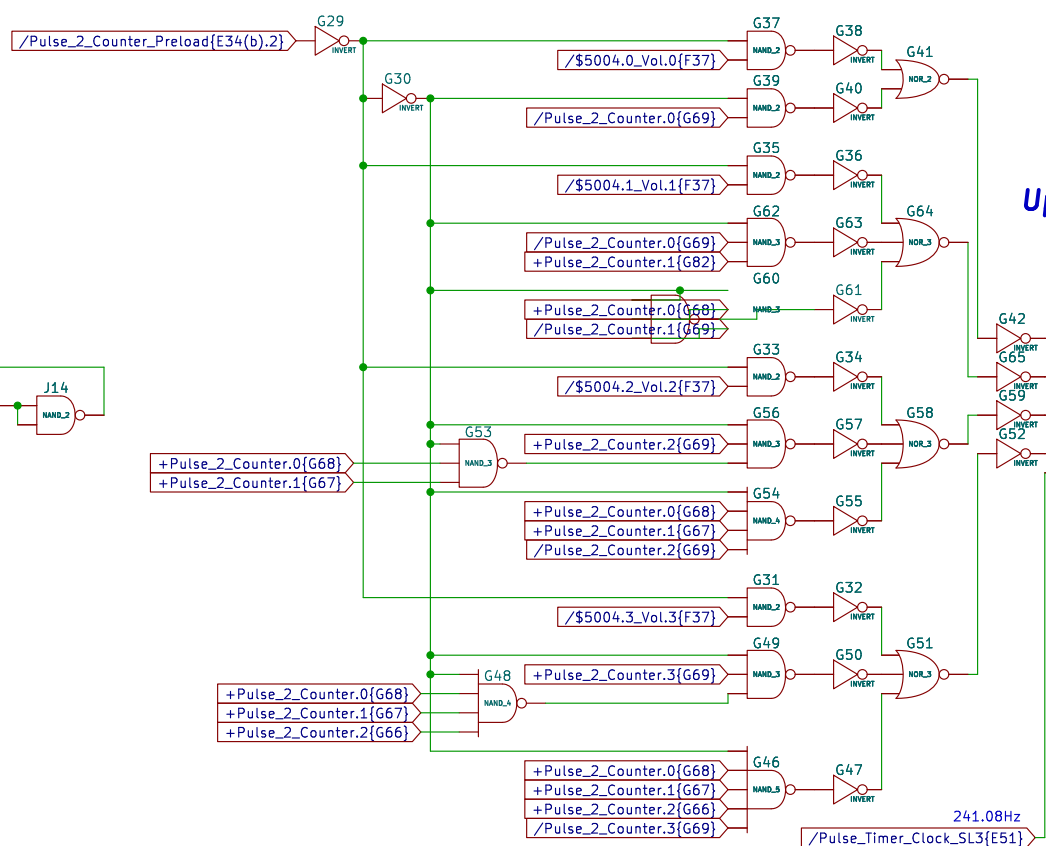


Pulse 1
Unused 4-bit Binary
Up-Counter With Preload

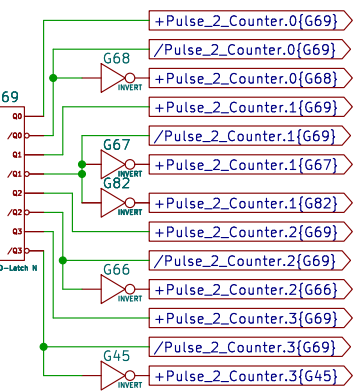


The only output that goes elsewhere is H78, which gets latched by H82(a), coming directly back here as /Pulse_1_Counter_Preload; all self-contained. It does not appear to provide any other external signals, so I don't think it actually does anything. It seems like an abandoned envelope counter that was left in.

It latches the 4-bit pulse volume, then counts up at a rate of 240 Hz. Nothing stops it from counting or rolling over from \$F back to \$0.



Pulse 2
Unused 4-bit Binary
Up-Counter With Preload



Sheet: /Artifacts/
File: artifacts.kicad_sch
Title:
Size: USLdger Date:
KiCad E.D.A. 9.0.7
Rev:
Id: 26/26